

Main_Solution

April 4, 2025

```
[3]: !pip install nbconvert
      !sudo apt install pandoc # or brew install pandoc on macOS

      !jupyter nbconvert --to pdf Main_Solution.ipynb
```

Requirement already satisfied: nbconvert in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (7.16.6)
Requirement already satisfied: beautifulsoup4 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from nbconvert) (4.13.3)
Requirement already satisfied: bleach!=5.0.0 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from bleach[css]!=5.0.0->nbconvert) (6.2.0)
Requirement already satisfied: defusedxml in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from nbconvert) (0.7.1)
Requirement already satisfied: jinja2>=3.0 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from nbconvert) (3.1.5)
Requirement already satisfied: jupyter-core>=4.7 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from nbconvert) (5.7.2)
Requirement already satisfied: jupyterlab-pygments in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from nbconvert) (0.3.0)
Requirement already satisfied: markupsafe>=2.0 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from nbconvert) (3.0.2)
Requirement already satisfied: mistune<4,>=2.0.3 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from nbconvert) (3.1.2)
Requirement already satisfied: nbclient>=0.5.0 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from nbconvert) (0.10.2)
Requirement already satisfied: nbformat>=5.7 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from nbconvert) (5.10.4)
Requirement already satisfied: packaging in /home/leandro-

sartini/miniconda3/envs/work/lib/python3.10/site-packages (from nbconvert) (24.2)

Requirement already satisfied: pandocfilters>=1.4.1 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from nbconvert) (1.5.0)

Requirement already satisfied: pygments>=2.4.1 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from nbconvert) (2.15.1)

Requirement already satisfied: traitlets>=5.1 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from nbconvert) (5.14.3)

Requirement already satisfied: webencodings in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert) (0.5.1)

Requirement already satisfied: tinycss2<1.5,>=1.1.0 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from bleach[css]!=5.0.0->nbconvert) (1.4.0)

Requirement already satisfied: platformdirs>=2.5 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from jupyter-core>=4.7->nbconvert) (4.3.6)

Requirement already satisfied: typing-extensions in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from mistune<4,>=2.0.3->nbconvert) (4.12.2)

Requirement already satisfied: jupyter-client>=6.1.12 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from nbclient>=0.5.0->nbconvert) (7.4.9)

Requirement already satisfied: fastjsonschema>=2.15 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from nbformat>=5.7->nbconvert) (2.21.1)

Requirement already satisfied: jsonschema>=2.6 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from nbformat>=5.7->nbconvert) (4.23.0)

Requirement already satisfied: soupsieve>1.2 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from beautifulsoup4->nbconvert) (2.5)

Requirement already satisfied: attrs>=22.2.0 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (25.1.0)

Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (2024.10.1)

Requirement already satisfied: referencing>=0.28.4 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.36.2)

Requirement already satisfied: rpds-py>=0.7.1 in /home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.22.3)

Requirement already satisfied: entrypoints in /home/leandro-

```

sartini/miniconda3/envs/work/lib/python3.10/site-packages (from jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert) (0.4)
Requirement already satisfied: nest-asyncio>=1.5.4 in /home/leandro-
sartini/miniconda3/envs/work/lib/python3.10/site-packages (from jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert) (1.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /home/leandro-
sartini/miniconda3/envs/work/lib/python3.10/site-packages (from jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert) (2.9.0.post0)
Requirement already satisfied: pyzmq>=23.0 in /home/leandro-
sartini/miniconda3/envs/work/lib/python3.10/site-packages (from jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert) (26.2.0)
Requirement already satisfied: tornado>=6.2 in /home/leandro-
sartini/miniconda3/envs/work/lib/python3.10/site-packages (from jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert) (6.4.2)
Requirement already satisfied: six>=1.5 in /home/leandro-
sartini/miniconda3/envs/work/lib/python3.10/site-packages (from python-
dateutil>=2.8.2->jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (1.16.0)
[sudo] password for leandro-sartini: [NbConvertApp] Converting notebook
Main_Solution.ipynb to pdf
[NbConvertApp] ERROR | Error while converting 'Main_Solution.ipynb'
Traceback (most recent call last):
  File "/home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-
packages/nbconvert/nbconvertapp.py", line 487, in export_single_notebook
    output, resources = self.exporter.from_filename(
  File "/home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-
packages/nbconvert/exporters/templateexporter.py", line 390, in from_filename
    return super().from_filename(filename, resources, **kw) #
type:ignore[return-value]
  File "/home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-
packages/nbconvert/exporters/exporter.py", line 201, in from_filename
    return self.from_file(f, resources=resources, **kw)
  File "/home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-
packages/nbconvert/exporters/templateexporter.py", line 396, in from_file
    return super().from_file(file_stream, resources, **kw) #
type:ignore[return-value]
  File "/home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-
packages/nbconvert/exporters/exporter.py", line 220, in from_file
    return self.from_notebook_node(
  File "/home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-
packages/nbconvert/exporters/pdf.py", line 184, in from_notebook_node
    latex, resources = super().from_notebook_node(nb, resources=resources, **kw)
  File "/home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-
packages/nbconvert/exporters/latex.py", line 92, in from_notebook_node
    return super().from_notebook_node(nb, resources, **kw)
  File "/home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-
packages/nbconvert/exporters/templateexporter.py", line 429, in
from_notebook_node
    output = self.template.render(nb=nb_copy, resources=resources)

```

```

File "/home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-
packages/jinja2/environment.py", line 1295, in render
    self.environment.handle_exception()
File "/home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-
packages/jinja2/environment.py", line 942, in handle_exception
    raise rewrite_traceback_stack(source=source)
File "/home/leandro-sartini/miniconda3/envs/work/share/jupyter/nbconvert/templ
ates/latex/index.tex.j2", line 8, in top-level template code
    ((* extends cell_style *))
File "/home/leandro-sartini/miniconda3/envs/work/share/jupyter/nbconvert/templ
ates/latex/style_jupyter.tex.j2", line 176, in top-level template code
    \prompt{(((prompt)))}{(((prompt_color)))}{(((execution_count)))}{(((extra_sp
ace)))}
File "/home/leandro-sartini/miniconda3/envs/work/share/jupyter/nbconvert/templ
ates/latex/base.tex.j2", line 7, in top-level template code
    ((*- extends 'document_contents.tex.j2' -*))
File "/home/leandro-sartini/miniconda3/envs/work/share/jupyter/nbconvert/templ
ates/latex/document_contents.tex.j2", line 51, in top-level template code
    ((*- block figure scoped -*))
File "/home/leandro-sartini/miniconda3/envs/work/share/jupyter/nbconvert/templ
ates/latex/display_priority.j2", line 5, in top-level template code
    ((*- extends 'null.j2' -*))
File "/home/leandro-
sartini/miniconda3/envs/work/share/jupyter/nbconvert/templates/latex/null.j2",
line 30, in top-level template code
    ((*- block body -*))
File "/home/leandro-sartini/miniconda3/envs/work/share/jupyter/nbconvert/templ
ates/latex/base.tex.j2", line 241, in block 'body'
    ((( super() )))
File "/home/leandro-
sartini/miniconda3/envs/work/share/jupyter/nbconvert/templates/latex/null.j2",
line 32, in block 'body'
    ((*- block any_cell scoped -*))
File "/home/leandro-
sartini/miniconda3/envs/work/share/jupyter/nbconvert/templates/latex/null.j2",
line 85, in block 'any_cell'
    ((*- block markdowncell scoped-*) ((*- endblock markdowncell -*))
File "/home/leandro-sartini/miniconda3/envs/work/share/jupyter/nbconvert/templ
ates/latex/document_contents.tex.j2", line 68, in block 'markdowncell'
    ((( cell.source | citation2latex | strip_files_prefix |
convert_pandoc('markdown+tex_math_double_backslash', 'json',extra_args=[]) |
resolve_references | convert_explicitly_relative_paths |
convert_pandoc('json','latex'))))
File "/home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-
packages/nbconvert/filters/pandoc.py", line 36, in convert_pandoc
    return pandoc(source, from_format, to_format, extra_args=extra_args)
File "/home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-
packages/nbconvert/utils/pandoc.py", line 50, in pandoc

```

```

    check_pandoc_version()
File "/home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-
packages/nbconvert/utils/pandoc.py", line 98, in check_pandoc_version
    v = get_pandoc_version()
File "/home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-
packages/nbconvert/utils/pandoc.py", line 75, in get_pandoc_version
    raise PandocMissing()
nbconvert.utils.pandoc.PandocMissing: Pandoc wasn't found.
Please check that pandoc is installed:
https://pandoc.org/installing.html

```

1 MECE

Model	Tasks and Comments	Status	Individual Responsible
Causal Trans-former based models using pretrained word em-beddings and position embed-dings	Preprocessing Steps - Drop null values, handle special chars, emojis, urls	Done	Aman/Prakash
"	Training - Model built, train and test handled correctly?	Done	Aman/Prakash
"	Evaluation - Cosine Similarity?	Done	Aman/Prakash
"	Interpretation using Lime	Done	Aman/Prakash
"	1st round of tuning	Done	Aman/Prakash
"	Next steps Recommended?	Improve response time using batch processing and model optimizations	Aman/Prakash

Model	Tasks and Comments	Status	Individual Responsible
Non-causal Trans-former based encoder-decoder model using embedding layer and relative positions with position encodings	Preprocessing Steps - 1. Step 1, 2. Step 2, 3. ...	Done/Pending	
”	Training - Model built, train and test handled correctly?	Done	Kundan
”	Evaluation - ROUGE-L Score, BERT Score?	Done/Pending	
”	Interpretation using Lime	Done/Pending	
”	1st round of tuning - What was the issue faced/tuned?	Done/Pending	
”	2nd round of tuning - What was the issue faced/tuned?	Done/Pending	
”	Final AUC value? Next steps Recommended?	Done/Pending	
Transfer learnt model built on a pretrained LLM such as GPT-2	Preprocessing Steps	Pending	
”	Training - Model built, train and test handled correctly?	Done	Jissy/Anjitha
”	Evaluation - ROUGE-L Score, BERT Score?	Done	Jissy/Anjitha
”	Interpretation using Lime	Done	Jissy/Anjitha
”	1st round of tuning - What was the issue faced/tuned?	Pending	
”	2nd round of tuning - What was the issue faced/tuned?	Pending	
”	Final AUC value? Next steps Recommended?	Pending	

Model	Tasks and Comments	Status	Individual Responsible
Using RAG approach on an open source LLM such as Llama or PALM or Gemini with a vector db using LangChain, DSPy etc.	Preprocessing Steps 1-Drop all Nan values on the 3 columns of the dataset. 2. Convert to documents, split on chunks of 500 words with an overlap of 50. 3-convert chunks to embeddings.	Done	Rodrigo Velazquez/ Leandro Sartini
”	Training - Model built, train and test handled correctly?	Done	Rodrigo Velazquez/Leandro Sartini
”	Evaluation - ROUGE-L Score, BERT Score?	Done: ROUGE-L F1 Score: 0.20 / BERTScore F1: 0.85, MAP@2: 0.13 / MRR@2: 0.60	Rodrigo Velazquez/Leandro Sartini
”	Interpretation using Lime	Done	Rodrigo Velazquez/Leandro Sartini

Model	Tasks and Comments	Status	Individual Responsible
”	1st round of tuning - What was the issue faced/tuned?	Done, tried to add separators to improve corpus chunking, but that made the model extremely slow, MAP and MRR were very small so I tried to play with the chunk size and overlap. Also tried Few Shot and CoT but only improved a little bit with few shot.	Rodrigo Velazquez/Leandro Sartini
”	2nd round of tuning - What was the issue faced/tuned?	Done, after improving a little with different chunk parameters and few shot, I tried to adjust temperature and the number of retrieved documents, which improved MAP and MRR a lot and also Rouge a little.	Rodrigo Velazquez/Leandro Sartini

Model	Tasks and Comments	Status	Individual Responsible
”	Final Score? Next steps Recommended?	MAP:0.15 MRR:0.7 BERT:0.89 ROUGE:0.39, we could improve a little the overall prompting on the new model as well.	Rodrigo Velazquez/Leandro Sartini
Using prompt en- gineering techniques (such as Few Shot, CoT, DSP etc) on a pretrained LLM	Preprocessing Steps - 1. Drop all Nan values 2. Removes spaces, tabs, newlines, or any other invisible characters from the beginning and end of each string	Done	Bibek / Jisna
	Preprocessing Steps - Cleaning, Tokenization, Formatting	Done	Jisna/Bibek
	Training - Model built, train and test handled correctly?	Done	Jisna/Bibek
	Evaluation - ROUGE-L Score, BERT Score?	Done. For Few Shots: ROUGE-L: 0.12891, BERT Score: 0.72868. For COTS: ROUGE-L: 0.07816 / BERT F1: 0.67336	Jisna/Bibek
	Interpretation using Lime	Done	Bibek

Model	Tasks and Comments	Status	Individual Responsible
	1st round of tuning - What was the issue faced/tuned?	Done. Tried reducing temperature and top_p for precision but due to limit daily usage of groq, used less dataset for tuning and only performed few shots. ROUGE-L F1: 0.1334, BERT F1 Score: 0.72876	Bibek
	2nd round of tuning - What was the issue faced/tuned?	Done. Tried increasing slightly temperature and top_p for precision but due to limit daily usage of groq, used less dataset for tuning and only performed few shots. ROUGE-L F1: 0.1337, BERT F1 Score: 0.72832	Bibek

Final AUC value? Next
steps Recommended?

Done, Next
Steps:
Optimize -
Improve
response time
using batch
processing
and model
optimizations

Bibek

2 Question 4

3 Imports

```
[1]: #!/pip install langchain faiss-cpu transformers sentence-transformers_
      ↪ctransformers pandas
#!/pip install -U langchain-community
import pandas as pd
from langchain.schema import Document
from langchain.text_splitter import RecursiveCharacterTextSplitter
#!/pip install -U langchain-huggingface
from langchain_huggingface import HuggingFaceEmbeddings
from langchain.vectorstores import FAISS
from langchain.llms import CTransformers
from langchain.chains import RetrievalQA

#!/pip install evaluate
from evaluate import load
#!/pip install rouge_score
#!/pip install bert_score
import pandas as pd
from tqdm import tqdm

import difflib
import numpy as np
from sklearn.metrics import average_precision_score
from tqdm import tqdm

from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain, RetrievalQA
import logging
logging.getLogger("transformers.modeling_utils").setLevel(logging.ERROR)
```

2025-04-02 17:51:10.755560: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them

```

off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-04-02 17:51:10.972061: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1743630671.052736 688604 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1743630671.079246 688604 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1743630671.267440 688604 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1743630671.267493 688604 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1743630671.267494 688604 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1743630671.267495 688604 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
2025-04-02 17:51:11.285558: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.

```

4 Load documents and create DB

```

[2]: # === STEP 1: load CSV ===
df_train = pd.read_csv("train.csv")
df_train = df_train.dropna(subset=["finalpassage", "query", "answers"])

df_val = pd.read_csv("valid.csv")
df_val = df_val.dropna(subset=["finalpassage", "query", "answers"])

df_all = pd.concat([df_train, df_val], ignore_index=True)
df_all["finalpassage"] = df_all["finalpassage"].astype(str)

# === Step 2: Convert 'finalpassage' to docs ===
docs = [
    Document(
        page_content=row["finalpassage"]
    )
]

```

```

    )
    for _, row in df_all.iterrows()
]

```

```

[34]: # === STEP 3: split docs in chunks ===
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
split_docs = splitter.split_documents(docs)

```

```

[35]: split_docs[:1]

```

```

[35]: [Document(metadata={}, page_content='At the same time, despite claiming the
review demonstrates a link between playing violent video games and aggression,
the authors acknowledged that some studies were inconsistent and that present
research is insufficient to establish whether this can lead to criminal violence
or delinquency.Kids who are bipolar, in their manic stages, very frequently
become aggressive. They lose self-control, they become impulsive. On the other
end of the spectrum, when they become depressed, although aggression')]

```

```

[36]: # === STEP 4: Generate embeddings ===
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/
↪all-MiniLM-L6-v2")

```

```

[37]: # === STEP 5: Create and save the vector database (FAISS) ===
vector_db = FAISS.from_documents(split_docs, embeddings)
vector_db.save_local("vectorial_faiss_db")

```

5 Load local Model

```

[38]: # === STEP 6: load LLaMA light with CTransformers ===
llm = CTransformers(
    model="llama-2-7b-chat.Q4_K_M.gguf",
    model_type="llama",
    config={"max_new_tokens": 512, "temperature": 0.5, "context_length": 1200}
)

```

huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid deadlocks...

To disable this warning, you can either:

- Avoid using `tokenizers` before the fork if possible
- Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

6 RAG

```
[39]: # === STEP 7: Create RAG chain (Retriever + LLM) ===
retriever = vector_db.as_retriever(search_kwargs={"k": 2})
qa_chain = RetrievalQA.from_chain_type(llm=llm, retriever=retriever,
    ↪chain_type="stuff") # we can play with the chain_type later using "refine"
    ↪or "map_rerank"
```

```
[40]: # === STEP 8: test a query ===
query = "why do children get aggressive"
output = qa_chain.invoke(query)
answer = output["result"]
print(f"Question: {query}\nAnswer: {answer}")
```

Question: why do children get aggressive

Answer: Based on the provided context, it seems that children may become aggressive due to frustration caused by blocking of goal-directed behavior. According to the frustration-aggression theory (1939), frustration creates a motive for aggression. Additionally, fear of punishment or disapproval may cause the aggressive behavior to be displaced against some other target, or oneself. Therefore, it is possible that children become aggressive when they are unable to achieve their goals or when they feel threatened by potential consequences.

let's explore the 2 documents it used for the context to see if the response makes sense

```
[41]: docs = retriever.get_relevant_documents(query)

for i, doc in enumerate(docs):
    print(f"\n--- Documento {i+1} ---")
    print(doc.page_content)
```

--- Documento 1 ---

when they become depressed, although aggression is less common, they can become irritable, and sometimes that irritability and cantankerousness causes kids to lash out.

--- Documento 2 ---

Psychological Influences of Aggressive Behavior. Frustration: the blocking of goal-directed behavior. The frustration-aggression theory (1939) : Frustration creates a motive for aggression. Fear of punishment or disapproval may cause the aggressive behavior to be displaced against some other target, or oneself. The amygdala in humans is the brain structure which has been linked to aggressive behavior. Aggressive behavior is genetically influenced: By selective breeding, aggressive and passive

```
[42]: # === STEP 8.1: test other query ===
query = "what county is grand rapids"
output = qa_chain.invoke(query)
```

```
answer = output["result"]
print(f"Question: {query}\nAnswer: {answer}")
```

Question: what county is grand rapids

Answer: The answer to the question "what county is Grand Rapids" can be found in the provided context. According to the text, Grand Rapids is located in Kent County.

Maybe we can tune the RAG prompt so it doesn't mention the context provided, just reply with the answer. This will be tested using Few-shot

7 Validation

7.0.1 ROUGE-L and BERT-F1

```
[52]: # load and sample validation dataset
df_val = pd.read_csv("valid.csv")
df_val = df_val.dropna(subset=["finalpassage", "query", "answers"])

df_val = df_val.sample(frac=0.005, random_state=42).reset_index(drop=True)
df_val = df_val[:10]
#df_val["query"] = df_val["query"].astype(str)
#df_val["answers"] = df_val["answers"].astype(str)

# Metrics
rouge = load("rouge")
bert_score = load("bertscore")

predictions = []
expected_responses = []

for _, row in tqdm(df_val.iterrows(), total=len(df_val), desc="Processing query's"):
    question = row["query"]
    real_answer = row["answers"]

    try:
        generated_response = qa_chain.invoke({"query": question})
    except Exception as e:
        print(f"Error with the question: {question} → {e}")
        generated_response = ""

    predictions.append(generated_response)
    expected_responses.append(real_answer)
```

Processing query's:

100% | 15/15

[09:18<00:00, 37.23s/it]

```
[53]: # Flatten predictions
predictions_text = [p["result"] if isinstance(p, dict) and "result" in p else
    ↪str(p) for p in predictions]
# Ensure responses are strings
expected_responses_texto = [str(r) for r in expected_responses]
```

```
[54]: # ROUGE-L
rouge_result = rouge.compute(predictions=predictions_text,
    ↪references=expected_responses_texto, rouge_types=["rougeL"])
print(f"\nROUGE-L F1 Score: {rouge_result['rougeL']:.4f}")

# BERT-F1
bert_result = bert_score.compute(
    predictions=predictions_text,
    references=expected_responses_texto,
    lang="en",
    device="cuda"
)
print(f" BERTScore F1 (average): {sum(bert_result['f1']) /
    ↪len(bert_result['f1']):.4f}")
```

ROUGE-L F1 Score: 0.2013

BERTScore F1 (average): 0.8505

- BERTScore F1 (Semantic Similarity (even if different words are used)): 0.85
- ROUGE-L: (Literal Overlap (exact words or phrases)) = 0.20
 - The model generates more “conversational” or summarized answers, which lowers ROUGE but not the BERTScore.

7.0.2 MAP and MRR with k=2

```
[89]: df_val["finalpassage"] = df_val["finalpassage"].astype(str)

k = 2 # retrieved documents

def is_relevant(retrieved_passages, gold_passage, threshold=0.85):
    """
    Compare each retrieved documents against expected document with fuzzy match.
    """
    for passage in retrieved_passages:
        ratio = difflib.SequenceMatcher(None, passage, gold_passage).ratio()
        if ratio >= threshold:
            return True
    return False

reciprocal_ranks = []
average_precisions = []
```



```

for _, row in tqdm(df_val.iterrows(), total=len(df_val), desc="Evaluating MAP/
↳MRR"):
    query = row["query"]
    expected_passage = row["finalpassage"]

    try:
        retrieved_docs = retriever.get_relevant_documents(query)
        retrieved_passages = [doc.page_content for doc in retrieved_docs[:k]]
    except Exception as e:
        print(f"Error on query: {query} → {e}")
        retrieved_passages = []

    # === MRR ===
    rank = None
    for i, passage in enumerate(retrieved_passages):
        ratio = difflib.SequenceMatcher(None, passage, expected_passage).ratio()
        if ratio >= 0.6:
            rank = i + 1
            break
    if rank:
        reciprocal_ranks.append(1 / rank)
    else:
        reciprocal_ranks.append(0)

    # === MAP ===
    # create binary relevance tags (1 if similar, 0 if no)
    relevance_scores = [
        1 if difflib.SequenceMatcher(None, passage, expected_passage).ratio()
↳>= 0.85 else 0
        for passage in retrieved_passages
    ]
    precision = average_precision_score(relevance_scores,
↳[1]*len(relevance_scores)) if any(relevance_scores) else 0
    average_precisions.append(precision)

# === Final Results ===
map_score = np.mean(average_precisions)
mrr_score = np.mean(reciprocal_ranks)

print(f"\n MAP (Mean Average Precision) @ {k}: {map_score:.4f}")
print(f" MRR (Mean Reciprocal Rank) @ {k}: {mrr_score:.4f}")

```

Evaluating MAP/MRR:

100%|

| 15/15

[00:01<00:00, 12.97it/s]

MAP (Mean Average Precision) @ 2: 0.1333
MRR (Mean Reciprocal Rank) @ 2: 0.6000

let's see the retrieved documents to check the reason for low values.

```
[57]: for idx, row in df_val[:5].iterrows():
    query = row["query"]
    expected_passage = row["finalpassage"]

    print(f"\n [{idx}] Query: {query}")
    print(f"Expected Passage (Ground Truth): {expected_passage[:100]}...")

    try:
        retrieved_docs = retriever.get_relevant_documents(query)
        retrieved_passages = [doc.page_content for doc in retrieved_docs[:k]]
        print(f"Retrieved Passages ({len(retrieved_passages)}):")
        for i, p in enumerate(retrieved_passages):
            sim = difflib.SequenceMatcher(None, p, expected_passage).ratio()
            print(f"  - Doc {i+1} (Sim: {sim:.3f}): {p[:100]}...")
    except Exception as e:
        print(f"Error retrieving docs for query: {e}")
        retrieved_passages = []

    # === MRR ===
    rank = None
    for i, passage in enumerate(retrieved_passages):
        ratio = difflib.SequenceMatcher(None, passage, expected_passage).ratio()
        if ratio >= 0.6:
            rank = i + 1
            break
    if rank:
        rr = 1 / rank
        reciprocal_ranks.append(rr)
        print(f"MRR: relevant doc found on position {rank} (RR={rr:.4f})")
    else:
        reciprocal_ranks.append(0)
        print("MRR: Did not find any relevant document")
```

[0] Query: what layer of skin creates heat insulator
Expected Passage (Ground Truth): Superficial heat. In contrast to deep heating modalities, superficial heating modalities usually do ...
Retrieved Passages (2):
- Doc 1 (Sim: 0.049): existing damage or to protect your healthy skin: keeping it well-guarded against damaging heat, chem...
- Doc 2 (Sim: 0.028): the blood vessels in your skin expand (dilate) to carry the excess heat to your skin's surface...

MRR: Did not find any relevant document

[1] Query: what is an average dose of GHB?

Expected Passage (Ground Truth): It's also one of the more famous "date rape" drugs, but most drug users who take GHB do so intention...

Retrieved Passages (2):

- Doc 1 (Sim: 0.026): per dose. The average dose is 1 to 5 grams and takes effect in 15 to 30 minutes, depending on the do...
- Doc 2 (Sim: 0.807): It's also one of the more famous "date rape" drugs, but most drug users who take GHB do so intention...

MRR: relevant doc found on position 2 (RR=0.5000)

[2] Query: who can blood group ab donate to

Expected Passage (Ground Truth): Blood group A individuals have the A antigen on the surface of their RBCs, and blood serum containin...

Retrieved Passages (2):

- Doc 1 (Sim: 0.652): Blood group A individuals have the A antigen on the surface of their RBCs, and blood serum containin...
- Doc 2 (Sim: 0.479): 1 Blood group B individuals have the B antigen on the surface of their RBCs, and blood serum contain...

MRR: relevant doc found on position 1 (RR=1.0000)

[3] Query: what is makaton used for

Expected Passage (Ground Truth): Makaton is a very simple language based on a list of simple everyday words, which uses speech, gestu...

Retrieved Passages (2):

- Doc 1 (Sim: 0.615): Makaton is a very simple language based on a list of simple everyday words, which uses speech, gestu...
- Doc 2 (Sim: 0.608): be used in and if possible give me two examples in the health care sector which it could be used..an...

MRR: relevant doc found on position 1 (RR=1.0000)

[4] Query: somatic therapy definition

Expected Passage (Ground Truth): Dance therapy or (Dance Movement Psychotherapy) also reflect something of this approach and are cons...

Retrieved Passages (2):

- Doc 1 (Sim: 0.499): in psychotherapies. Done by organic methods. SOMATIC THERAPY: Somatic therapy is treating mental dis...
- Doc 2 (Sim: 0.037): medical Definition of somatic 1 a: of, relating to, or affecting the body especially as distinguishe...

MRR: Did not find any relevant document

In general the first retrieved document is the one with higher similarity, also the document split in chunks can be eclipsing some of the answers and we might be retrieving good answers but chunked answers which decrease the similarity

7.1 Prompt templates

7.1.1 Few Shot

```
[92]: few_shot_prompt = """
You are a helpful, respectful and honest assistant. Always answer as helpfully
as possible, while being safe. Your answers should not include any harmful,
unethical, racist, sexist, toxic, dangerous, or illegal content. Please ensure
that your responses are socially unbiased and positive in nature. If a
question does not make any sense, or is not factually coherent, explain why
instead of answering something not correct. If you don't know the answer to a
question, please don't share false information.

Example 1:
Context: Kids who are bipolar, in their manic stages, very frequently become
↳aggressive.
Question: Why do children get aggressive?
Answer: Children with bipolar disorder can become aggressive due to impulsivity
↳during manic or depressive episodes.

Now, answer the following:

Context: {context}
Question: {question}
Answer:
"""
```

7.1.2 CoT Chain of Thought

```
[59]: cot_prompt = """
You are a helpful, respectful and honest assistant. Always answer as helpfully
as possible, while being safe. Your answers should not include any harmful,
unethical, racist, sexist, toxic, dangerous, or illegal content. Please ensure
that your responses are socially unbiased and positive in nature. If a
question does not make any sense, or is not factually coherent, explain why
instead of answering something not correct. If you don't know the answer to a
question, please don't share false information. Use the context below to
↳think step by step and answer the question.

Context: {context}

Question: {question}

Let's think step by step:
"""
```

7.2 let's start with Few Shot

```
[64]: def metrics(prompt_template):

    # Crear plantilla personalizada (ejemplo: Few-Shot)
    prompt_template = PromptTemplate(
        input_variables=["context", "question"],
        template=prompt_template
    )

    # Crear LLMChain con ese prompt
    qa_chain = RetrievalQA.from_chain_type(
        llm=llm,
        retriever=retriever,
        chain_type="stuff",
        chain_type_kwargs={"prompt": prompt_template}
    )

    # Metrics
    rouge = load("rouge")
    bert_score = load("bertscore")

    predictions = []
    expected_responses = []

    for _, row in tqdm(df_val[:10].iterrows(), total=len(df_val[:10]),
↳desc="Procesando consultas"):
        query = row["query"]
        real_answer = row["answers"]
        expected_passage = row["finalpassage"]

        try:
            generated_response = qa_chain.invoke({"query": query})
            retrieved_docs = retriever.get_relevant_documents(query)
            retrieved_passages = [doc.page_content for doc in retrieved_docs[:
↳k]]

            for i, p in enumerate(retrieved_passages):
                sim = difflib.SequenceMatcher(None, p, expected_passage).ratio()
        except Exception as e:
            print(f"Error for query: {query} → {e}")
            generated_response = ""
            retrieved_passages = []

    predictions.append(generated_response)
    expected_responses.append(real_answer)

    # === MRR ===
```

```

rank = None
for i, passage in enumerate(retrieved_passages):
    ratio = difflib.SequenceMatcher(None, passage, expected_passage).
↪ratio()
    if ratio >= 0.6:
        rank = i + 1
        break
if rank:
    rr = 1 / rank
    reciprocal_ranks.append(rr)
else:
    reciprocal_ranks.append(0)

# === MAP ===
# create binary relevance tags (1 if similar, 0 if no)
relevance_scores = [
    1 if difflib.SequenceMatcher(None, passage, expected_passage).
↪ratio() >= 0.85 else 0
    for passage in retrieved_passages
]
precision = average_precision_score(relevance_scores,
↪[1]*len(relevance_scores)) if any(relevance_scores) else 0
average_precisions.append(precision)

# Falten predictions
predictions_text = [p["result"] if isinstance(p, dict) and "result" in p
↪else str(p) for p in predictions]
# Ensure responses are strings
expected_responses_texto = [str(r) for r in expected_responses]

# ROUGE-L
rouge_result = rouge.compute(predictions=predictions_text,
↪references=expected_responses_texto, rouge_types=["rougeL"])
print(f"\n ROUGE-L F1 Score: {rouge_result['rougeL']:.4f}")

# BERT-F1
bert_result = bert_score.compute(
    predictions=predictions_text,
    references=expected_responses_texto,
    lang="en",
    device="cuda" # Usa GPU if available
)
print(f" BERTScore F1 (promedio): {sum(bert_result['f1']) /
↪len(bert_result['f1']):.4f}")

```

```
# === Final Results ===
map_score = np.mean(average_precisions)
mrr_score = np.mean(reciprocal_ranks)

print(f"\n MAP (Mean Average Precision) @ {k}: {map_score:.4f}")
print(f" MRR (Mean Reciprocal Rank) @ {k}: {mrr_score:.4f}")
```

```
[65]: metrics(few_shot_prompt)
```

```
Procesando consultas:
100%|                               | 10/10
[09:34<00:00, 57.50s/it]
```

```
ROUGE-L F1 Score: 0.1808
BERTScore F1 (promedio): 0.8532
```

```
MAP (Mean Average Precision) @ 2: 0.1200
MRR (Mean Reciprocal Rank) @ 2: 0.5758
```

```
[74]: predictions[2]
```

```
[74]: {'query': 'who can blood group ab donate to',
      'result': ' AB individuals can donate blood to individuals with type A or AB,
but they cannot receive blood from individuals of groups B or O (with B being
preferable).'}

```

The results were very similar to before: ROUGE-L F1 Score: 0.2013 | BERTScore F1: 0.8505, MAP@2: 0.1333 | MRR@2: 0.6000 we saw a small decrease on Rouge MAP and MRR.

7.3 Now for CoT

```
[68]: metrics(cot_prompt)
```

```
Procesando consultas:
100%|                               | 10/10
[12:02<00:00, 72.25s/it]
```

```
ROUGE-L F1 Score: 0.1520
BERTScore F1 (promedio): 0.8408
```

```
MAP (Mean Average Precision) @ 2: 0.1143
MRR (Mean Reciprocal Rank) @ 2: 0.5814
```

Yes, for rouge it makes sense that Rouge will decrease since now we are adding the step by step approach. ROUGE-L F1 Score: 0.2013 | BERTScore F1: 0.8505, MAP@2: 0.1333 | MRR@2: 0.6000 we saw a decrease on Rouge, BERT, MAP and MRR.

```
[75]: predictions[2]
```

```
[75]: {'query': 'who can blood group ab donate to',  
      'result': ' AB individuals can donate blood to individuals with type A or AB,  
but they cannot receive blood from individuals of groups B or O (with B being  
preferable).'}

[73]: expected_responses[2]
```

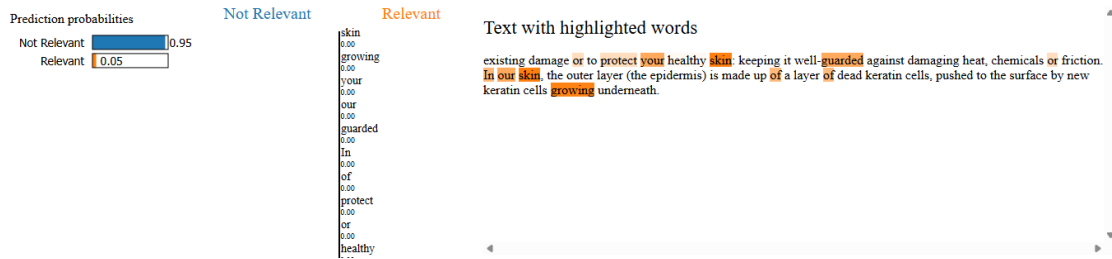
```
[73]: 'A and B'
```

8 LIME

We will check which words from the context influenced more the response.

```
[87]: import difflib  
  
def predict_relevance(contexts):  
    results = []  
    for c in contexts:  
        sim = difflib.SequenceMatcher(None, c, expected_passage).ratio()  
        results.append([1 - sim, sim]) # [prob_no_relevante, prob_relevante]  
    return np.array(results)  
  
[88]: from lime.lime_text import LimeTextExplainer  
  
# Usa un ejemplo real  
row = df_val.iloc[0]  
query = row["query"]  
expected_passage = row["finalpassage"]  
  
# Documento a analizar  
original_context = retriever.get_relevant_documents(query)[0].page_content[:512]  
  
explainer = LimeTextExplainer(class_names=["Not Relevant", "Relevant"])  
  
exp = explainer.explain_instance(  
    original_context,  
    predict_relevance,  
    num_features=10,  
    num_samples=10 # bajo para pruebas  
)  
  
exp.show_in_notebook(text=True)
```

<IPython.core.display.HTML object>



9 Tuning

For the tuning we can play with the temperature level, the number of K, the chain_type using “refine” or “map_rerank” , also the chunk of documents can be revised.

Baseline: ROUGE-L F1 Score: 0.2013 | BERTScore F1: 0.8505, MAP@2: 0.1333 | MRR@2: 0.6000

Get the biggest final passage and the average lenght to tune the chunk size

```
[104]: longest = 0
lengths = []
for _, row in df_val.iterrows():
    length = len(row["finalpassage"])
    lengths.append(length)
    if length > longest:
        longest = length
print(longest)
```

1177

```
[105]: suma = 0
for lens in lengths:
    suma += lens
average = suma/len(lengths)
average
```

[105]: 795.3333333333334

```
[107]: from statistics import mode

moda = mode(lengths)
moda
```

[107]: 828

Based on the average and the mode we will be using a chunk size of 800 with a 100 overlap

9.1 New metrics parametrized

```
[3]: def metrics(prompt_template):
    reciprocal_ranks = []
    average_precisions = []

    # Crear plantilla personalizada (ejemplo: Few-Shot)
    prompt_template = PromptTemplate(
        input_variables=["context", "question"],
        template=prompt_template
    )

    # Crear LLMChain con ese prompt
    qa_chain = RetrievalQA.from_chain_type(
        llm=llm,
        retriever=retriever_no,
        chain_type=chain_type,
        chain_type_kwargs={"prompt": prompt_template}
    )

    # Metrics
    rouge = load("rouge")
    bert_score = load("bertscore")

    predictions = []
    expected_responses = []

    for _, row in tqdm(df_val[:10].iterrows(), total=len(df_val[:10]),
↳desc="Procesing queries"):
        query = row["query"]
        real_answer = row["answers"]
        expected_passage = row["finalpassage"]

        try:
            print(f'try {query}')
            generated_response = qa_chain.invoke({"query": query})
            retrieved_docs = retriever_no.get_relevant_documents(query)
            retrieved_passages = [doc.page_content for doc in retrieved_docs[:
↳k]]

            for i, p in enumerate(retrieved_passages):
                sim = difflib.SequenceMatcher(None, p, expected_passage).ratio()
        except Exception as e:
            print(f"Error for query: {query} → {e}")
            generated_response = ""
            retrieved_passages = []

    predictions.append(generated_response)
```

```

expected_responses.append(real_answer)

    # === MRR ===
    rank = None
    for i, passage in enumerate(retrieved_passages):
        ratio = difflib.SequenceMatcher(None, passage, expected_passage).
↪ratio()
        if ratio >= 0.6:
            rank = i + 1
            break
    if rank:
        rr = 1 / rank
        reciprocal_ranks.append(rr)
    else:
        reciprocal_ranks.append(0)

    # === MAP ===
    # create binary relevance tags (1 if similar, 0 if no)
    relevance_scores = [
        1 if difflib.SequenceMatcher(None, passage, expected_passage).
↪ratio() >= 0.85 else 0
        for passage in retrieved_passages
    ]
    precision = average_precision_score(relevance_scores,
↪[1]*len(relevance_scores)) if any(relevance_scores) else 0
    average_precisions.append(precision)

    # Flatten predictions
    predictions_text = [p["result"] if isinstance(p, dict) and "result" in p
↪else str(p) for p in predictions]
    # Ensure responses are strings
    expected_responses_texto = [str(r) for r in expected_responses]

    # ROUGE-L
    rouge_result = rouge.compute(predictions=predictions_text,
↪references=expected_responses_texto, rouge_types=["rougeL"])
    print(f"\n ROUGE-L F1 Score: {rouge_result['rougeL']:.4f}")

    # BERT-F1
    bert_result = bert_score.compute(
        predictions=predictions_text,
        references=expected_responses_texto,
        lang="en",
        device="cuda" # Usa GPU if available
    )

```

```

    print(f" BERTScore F1 (promedio): {sum(bert_result['f1']) /
↪len(bert_result['f1']):.4f}")

    # === Final Results ===
    map_score = np.mean(average_precisions)
    mrr_score = np.mean(reciprocal_ranks)

    print(f"\n MAP (Mean Average Precision) @ {k}: {map_score:.4f}")
    print(f" MRR (Mean Reciprocal Rank) @ {k}: {mrr_score:.4f}")

```

9.2 Tuning Iterations

```

[6]: K = 2 # for retriever
k = 2
chunk_size=800
chunk_overlap=100
temperature = 0.5
prompt_template = '''
You are a helpful, respectful and honest assistant. Your answers should not
↪include any harmful,
unethical, racist, sexist, toxic, dangerous, or illegal content.
Answer the following:

Context: {context}
Question: {question}
Answer:
'''

chain_type="stuff" # we can play with the chain_type later using "refine" or
↪"map_rerank" separators=["\n\n", "\n", ".", " ", ""]

splitter = RecursiveCharacterTextSplitter(chunk_size=chunk_size,
↪chunk_overlap=chunk_overlap)
split_docs = splitter.split_documents(docs)
print('Docs split ....')
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/
↪all-MiniLM-L6-v2")
vector_db2 = FAISS.from_documents(split_docs, embeddings)
vector_db2.save_local("vectorial_faiss_db_2")
print('VectorDB created ....')
# llm = CTransformers(
#     model="llama-2-7b-chat.Q4_K_M.gguf",
#     model_type="llama",
#     config={"max_new_tokens": 512, "temperature": temperature,
↪"context_length": 1200}
# )

```

```
retriever_no = vector_db2.as_retriever(search_kwargs={"k": K})    # change the_
↪number of the vectordb name
metrics(prompt_template)
```

Docs split ...

VectorDB created ...

Procesing queries: 0%|

| 0/10 [00:00<?, ?it/s]

try how popular is the name conrad

Procesing queries: 10%|

| 1/10 [03:10<28:31, 190.14s/it]

try disney hakuna matata meaning

Procesing queries: 20%|

| 2/10 [03:53<13:50, 103.78s/it]

try what does emr stand for

Procesing queries: 30%|

| 3/10 [04:24<08:12, 70.37s/it]

try what is a dog's ruff

Procesing queries: 40%|

| 4/10 [04:59<05:40, 56.69s/it]

try what is the part on your arm where they draw blood called

Procesing queries: 50%|

| 5/10 [05:38<04:11, 50.30s/it]

try definition and explanation of singer songwriter

Procesing queries: 60%|

| 6/10 [07:01<04:04, 61.22s/it]

try where can chikungunya be found

Procesing queries: 70%|

| 7/10 [07:51<02:53, 57.79s/it]

try what is waterboarding

Procesing queries: 80%|

| 8/10 [08:47<01:54, 57.22s/it]

try what is an organism that captures energy and stores it in food as chemical energy

Procesing queries:

90%| | 9/10

[10:02<01:02, 62.63s/it]

try whats the 3rd book in warroirs power of thre

Processing queries:
100%| | 10/10
[10:29<00:00, 62.98s/it]

ROUGE-L F1 Score: 0.1817
BERTScore F1 (promedio): 0.8609

MAP (Mean Average Precision) @ 2: 0.4500
MRR (Mean Reciprocal Rank) @ 2: 0.7000

```
[ ]: ROUGE-L F1 Score: 0.2013 | BERTScore F1: 0.8505, MAP@2: 0.1333 | MRR@2: 0.6000
```

Now trying with $K = 3$

```
[7]: K = 3 # for retriever
k = 3
chunk_size=800
chunk_overlap=100
temperature = 0.5
prompt_template = '''
You are a helpful, respectful and honest assistant. Your answers should not
    ↪include any harmful,
unethical, racist, sexist, toxic, dangerous, or illegal content.
Answer the following:

Context: {context}
Question: {question}
Answer:
'''

chain_type="stuff" # we can play with the chain_type later using "refine" or
    ↪"map_rerank" separators=["\n\n", "\n", ".", " ", ""]

splitter = RecursiveCharacterTextSplitter(chunk_size=chunk_size,
    ↪chunk_overlap=chunk_overlap)
split_docs = splitter.split_documents(docs)
print('Docs split ....')
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/
    ↪all-MiniLM-L6-v2")
vector_db2 = FAISS.from_documents(split_docs, embeddings)
vector_db2.save_local("vectorial_faiss_db_2")
print('VectorDB created ....')
# llm = CTransformers(
#     model="llama-2-7b-chat.Q4_K_M.gguf",
#     model_type="llama",
#     config={"max_new_tokens": 512, "temperature": temperature,
#     ↪"context_length": 1200}
```

```
# )
retriever_no = vector_db2.as_retriever(search_kwargs={"k": K}) # change the
↪number of the vectordb name
metrics(prompt_template)
```

Docs split ...

VectorDB created ...

Procesing queries: 0%|

| 0/10 [00:00<?, ?it/s]

try how popular is the name conrad

Procesing queries: 10%|

| 1/10 [03:58<35:47, 238.63s/it]

try disney hakuna matata meaning

Procesing queries: 20%|

| 2/10 [05:28<20:09, 151.19s/it]

try what does emr stand for

Procesing queries: 30%|

| 3/10 [06:26<12:40, 108.59s/it]

try what is a dog's ruff

Procesing queries: 40%|

| 4/10 [07:33<09:13, 92.19s/it]

try what is the part on your arm where they draw blood called

Procesing queries: 50%|

| 5/10 [08:45<07:03, 84.78s/it]

try definition and explanation of singer songwriter

Procesing queries: 60%|

| 6/10 [09:57<05:21, 80.47s/it]

try where can chikungunya be found

Procesing queries: 70%|

| 7/10 [11:21<04:05, 81.69s/it]

try what is waterboarding

Procesing queries: 80%|

| 8/10 [12:50<02:48, 84.05s/it]

try what is an organism that captures energy and stores it in food as chemical energy

Procesing queries:

90%|

| 9/10

[14:23<01:26, 86.83s/it]

try whats the 3rd book in warroirs power of thre

Procesing queries:

100%| | 10/10
[14:53<00:00, 89.30s/it]

ROUGE-L F1 Score: 0.1666

BERTScore F1 (promedio): 0.8616

MAP (Mean Average Precision) @ 3: 0.3000

MRR (Mean Reciprocal Rank) @ 3: 0.7000

```
[ ]: ROUGE-L F1 Score: 0.2013 | BERTScore F1: 0.8505, MAP@2: 0.1333 | MRR@2: 0.6000  
Improvement vs baseline but previous results were better.
```

Lets play with the temperature

```
[8]: K = 2 # for retriever  
k = 2  
chunk_size=800  
chunk_overlap=100  
temperature = 0.3  
prompt_template = '''  
You are a helpful, respectful and honest assistant. Your answers should not  
    ↳include any harmful,  
unethical, racist, sexist, toxic, dangerous, or illegal content.  
Answer the following:  
  
Context: {context}  
Question: {question}  
Answer:  
'''  
  
chain_type="stuff" # we can play with the chain_type later using "refine" or  
    ↳"map_rerank" separators=["\n\n", "\n", ".", " ", ""]  
  
splitter = RecursiveCharacterTextSplitter(chunk_size=chunk_size,  
    ↳chunk_overlap=chunk_overlap)  
split_docs = splitter.split_documents(docs)  
print('Docs split ....')  
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/  
    ↳all-MiniLM-L6-v2")  
vector_db3 = FAISS.from_documents(split_docs, embeddings)  
vector_db3.save_local("vectorial_faiss_db_3")  
print('VectorDB created ....')  
# llm = CTransformers(  
#     model="llama-2-7b-chat.Q4_K_M.gguf",  
#     model_type="llama",
```



```
#      config={"max_new_tokens": 512, "temperature": temperature,
↳ "context_length": 1200}
# )
retriever_no = vector_db3.as_retriever(search_kwargs={"k": K}) # change the
↳ number of the vectordb name
metrics(prompt_template)
```

Docs split ...

VectorDB created ...

Procesing queries: 0%|

| 0/10 [00:00<?, ?it/s]

try how popular is the name conrad

Procesing queries: 10%|

| 1/10 [03:44<33:44, 224.95s/it]

try disney hakuna matata meaning

Procesing queries: 20%|

| 2/10 [04:31<15:58, 119.81s/it]

try what does emr stand for

Procesing queries: 30%|

| 3/10 [05:00<09:10, 78.60s/it]

try what is a dog's ruff

Procesing queries: 40%|

| 4/10 [05:38<06:13, 62.33s/it]

try what is the part on your arm where they draw blood called

Procesing queries: 50%|

| 5/10 [06:20<04:36, 55.27s/it]

try definition and explanation of singer songwriter

Procesing queries: 60%|

| 6/10 [09:13<06:20, 95.20s/it]

try where can chikungunya be found

Procesing queries: 70%|

| 7/10 [10:04<04:02, 80.76s/it]

try what is waterboarding

Procesing queries: 80%|

| 8/10 [11:06<02:29, 74.61s/it]

try what is an organism that captures energy and stores it in food as chemical energy

```
Processing queries:
90%|                               | 9/10
[12:06<01:10, 70.18s/it]
```

try whats the 3rd book in warroirs power of thre

```
Processing queries:
100%|                              | 10/10
[12:32<00:00, 75.24s/it]
```

```
ROUGE-L F1 Score: 0.2133
BERTScore F1 (promedio): 0.8640
```

```
MAP (Mean Average Precision) @ 2: 0.4500
MRR (Mean Reciprocal Rank) @ 2: 0.7000
```

```
[ ]: ROUGE-L F1 Score: 0.2013 | BERTScore F1: 0.8505, MAP@2: 0.1333 | MRR@2: 0.6000
    ↳so far these have been the best parameters.
```

9.3 last tune

Now let's try one last with "map_reduce" to apply the model to each document and then combine them. # This was taking to much time so it's not a feasible solution. Instead let's try a lower value of temperature since that shifted the most our metrics.

```
[6]: K = 2 # for retriever
k = 2
chunk_size=800
chunk_overlap=100
temperature = 0.1
prompt_template = '''
You are a helpful, respectful and honest assistant. Your answers should not
↳include any harmful,
unethical, racist, sexist, toxic, dangerous, or illegal content.
Answer the following:

Context: {context}
Question: {question}
Answer:
'''

chain_type="stuff" # we can play with the chain_type later using "refine" or
↳"map_rerank" separators=["\n\n", "\n", ".", " ", ""]

splitter = RecursiveCharacterTextSplitter(chunk_size=chunk_size,
↳chunk_overlap=chunk_overlap)
split_docs = splitter.split_documents(docs)
print('Docs split ....')
```

```

embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/
↳all-MiniLM-L6-v2")
vector_db3 = FAISS.from_documents(split_docs, embeddings)
vector_db3.save_local("vectorial_faiss_db_3")
print('VectorDB created ....')
llm = CTransformers(
    model="llama-2-7b-chat.Q4_K_M.gguf",
    model_type="llama",
    config={"max_new_tokens": 512, "temperature": temperature, "context_length":
↳ 1200}
)
# vector_db3 = FAISS.load_local("vectorial_faiss_db_3",
↳HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2"),
↳allow_dangerous_deserialization = True)
retriever_no = vector_db3.as_retriever(search_kwargs={"k": K}) # change the
↳number of the vectordb name
metrics(prompt_template)

```

Docs split ...

VectorDB created ...

huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid deadlocks...

To disable this warning, you can either:

- Avoid using `tokenizers` before the fork if possible
- Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

Procesing queries: 0%|

| 0/10 [00:00<?, ?it/s]

try how popular is the name conrad

Procesing queries: 10%|

| 1/10 [02:09<19:27, 129.70s/it]

try disney hakuna matata meaning

Procesing queries: 20%|

| 2/10 [03:04<11:25, 85.65s/it]

try what does emr stand for

Procesing queries: 30%|

| 3/10 [03:37<07:11, 61.63s/it]

try what is a dog's ruff

Procesing queries: 40%|

| 4/10 [04:19<05:22, 53.78s/it]

try what is the part on your arm where they draw blood called

```

Procesing queries: 50%|
| 5/10 [05:10<04:24, 52.88s/it]

try definition and explanation of singer songwriter

Procesing queries: 60%|
| 6/10 [11:43<11:14, 168.51s/it]

try where can chikungunya be found

Procesing queries: 70%|
| 7/10 [12:39<06:35, 131.85s/it]

try what is waterboarding

Procesing queries: 80%|
| 8/10 [13:45<03:41, 110.59s/it]

try what is an organism that captures energy and stores it in food as chemical
energy

Procesing queries:
90%|                                     | 9/10
[14:47<01:35, 95.66s/it]

try whats the 3rd book in warroirs power of thre

Procesing queries:
100%|                                     | 10/10
[15:12<00:00, 91.25s/it]

```

```

ROUGE-L F1 Score: 0.2013
BERTScore F1 (promedio): 0.8642

```

```

MAP (Mean Average Precision) @ 2: 0.4500
MRR (Mean Reciprocal Rank) @ 2: 0.7000

```

Stil the previous results were better for ROUGE and almost the same for the other metrics.

9.4 Adding leo's run with the new model as well.

9.4.1 Loading and preparing the vector base and finalpassage

```

[2]: # === STEP 1: load CSV ===
df_train = pd.read_csv("data/train.csv")
df_train = df_train.dropna(subset=["finalpassage", "query", "answers"])

df_val = pd.read_csv("data/valid.csv")
df_val = df_val.dropna(subset=["finalpassage", "query", "answers"])

df_all = pd.concat([df_train, df_val], ignore_index = True)
df_all["finalpassage"] = df_all["finalpassage"].astype(str)

```

```

# === Step 2: Convert 'finalpassage' to docs ===
docs = [
    Document(
        page_content=row["finalpassage"]
    )
    for _, row in df_all.iterrows()
]

# === STEP 3: split docs in chunks ===
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
split_docs = splitter.split_documents(docs)

split_docs[:1]

# === STEP 4: Generate embeddings ===
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/
↪all-MiniLM-L6-v2")

# === STEP 5: Create and save the vector database (FAISS) ===
vector_db = FAISS.from_documents(split_docs, embeddings)
vector_db.save_local("vectorial_faiss_db")

```

```

/home/leandro-sartini/miniconda3/envs/work/lib/python3.10/site-
packages/torch/utils/_pytree.py:185: FutureWarning: optree is installed but the
version is too old to support PyTorch Dynamo in C++ pytree. C++ pytree support
is disabled. Please consider upgrading optree using `python3 -m pip install
--upgrade 'optree>=0.13.0'`.
  warnings.warn(
2025-04-02 22:33:21.224595: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-04-02 22:33:21.294295: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1743647601.318440 44964 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1743647601.325805 44964 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1743647601.369710 44964 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1743647601.369731 44964 computation_placer.cc:177] computation

```

placer already registered. Please check linkage and avoid linking the same target more than once.
W0000 00:00:1743647601.369732 44964 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.
W0000 00:00:1743647601.369734 44964 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.
2025-04-02 22:33:21.379493: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

9.4.2 Loading Llama

```
[3]: from langchain.llms import CTransformers

llm = CTransformers(
    model="models/mistral-7b-instruct-v0.1.Q4_K_M.gguf",
    model_type="llama",
    config={
        "max_new_tokens": 512,
        "temperature": 0.6,
        "context_length": 2048,
        "gpu_layers": 33, # Max layers to offload to GPU (auto-manages based
        ↪ on VRAM)
    }
)

[4]: retriever = vector_db.as_retriever(search_kwargs={"k": 3}) # Retrieve top 3
    ↪ chunks

[5]: def rag_query(llm, retriever, query: str) -> str:
    docs = retriever.get_relevant_documents(query)
    context = "\n".join(doc.page_content for doc in docs)

    prompt = f"""You are a helpful assistant. Use the context below to answer
    ↪ the user's question.

Context:
{context}

Question:
{query}

Answer: """
```

```
return llm(prompt)
```

```
[6]: # === STEP 7: Create RAG chain (Retriever + LLM) ===
retriever = vector_db.as_retriever(search_kwargs={"k": 2})
qa_chain = RetrievalQA.from_chain_type(llm=llm, retriever=retriever,
    ↪chain_type="stuff") # we can play with the chain_type later using "refine"
    ↪or "map_rerank"

[7]: # === STEP 8: test a query ===
query = "definition of what the skeletal"
output = qa_chain.invoke(query)
answer = output["result"]
print(f"Question: {query}\nAnswer: {answer}")
```

Question: definition of what the skeletal

Answer: The skeletal system includes all of the bones and joints in the body. Each bone is a complex living organ that is made up of many cells, protein fibers, and minerals. The skeleton acts as a scaffold by providing support and protection for the soft tissues that make up the rest of the body. The skeletal system also provides attachment points for muscles to allow movements at the joints. New blood cells are produced by the red bone marrow inside of our bones.

9.5 Let's check which other documents it brings

```
[8]: docs = retriever.get_relevant_documents(query)

for i, doc in enumerate(docs):
    print(f"\n--- Document {i+1} ---")
    print(doc.page_content)
```

--- Document 1 ---

The skeletal system includes all of the bones and joints in the body. Each bone is a complex living organ that is made up of many cells, protein fibers, and minerals. The skeleton acts as a scaffold by providing support and protection for the soft tissues that make up the rest of the body. The skeletal system also provides attachment points for muscles to allow movements at the joints. New blood cells are produced by the red bone marrow inside of our bones. The axial skeleton is the part of the

--- Document 2 ---

The skeletal system includes all of the bones and joints in the body. Each bone is a complex living organ that is made up of many cells, protein fibers, and minerals. The skeleton acts as a scaffold by providing support and protection for the soft tissues that make up the rest of the body. The skeletal system supports and protects the body while giving it shape and form. This system is composed of connective tissues including bone, cartilage, tendons, and

ligaments. Nutrients are provided to this

```
/tmp/ipykernel_44964/4275237952.py:1: LangChainDeprecationWarning: The method
`BaseRetriever.get_relevant_documents` was deprecated in langchain-core 0.1.46
and will be removed in 1.0. Use :meth:`~invoke` instead.
```

```
docs = retriever.get_relevant_documents(query)
```

9.5.1 In this case it's just bringing the same document

10 Validation

10.0.1 ROUGE-L and BERT-F1

```
[18]: # load and sample validation dataset
df_val = pd.read_csv("data/valid.csv")
df_val = df_val.dropna(subset=["finalpassage", "query", "answers"])

df_val = df_val.sample(frac=0.005, random_state=42).reset_index(drop=True)
df_val = df_val[:10]
#df_val["query"] = df_val["query"].astype(str)
#df_val["answers"] = df_val["answers"].astype(str)

# Metrics
rouge = load("rouge")
bert_score = load("bertscore")

predictions = []
expected_responses = []

for _, row in tqdm(df_val.iterrows(), total=len(df_val), desc="Procesing_
↳query's"):
    question = row["query"]
    real_answer = row["answers"]

    try:
        generated_response = qa_chain.invoke({"query": question})
    except Exception as e:
        print(f"Error with the question: {question} → {e}")
        generated_response = ""

    predictions.append(generated_response)
    expected_responses.append(real_answer)
```

Procesing query's:

100% | 10/10

[01:19<00:00, 7.98s/it]

```
[19]: # Falten predictions
```



```

predictions_text = [p["result"] if isinstance(p, dict) and "result" in p else
↳str(p) for p in predictions]
# Ensure responses are strings
expected_responses_texto = [str(r) for r in expected_responses]

```

```

[20]: # ROUGE-L
rouge_result = rouge.compute(predictions=predictions_text,
↳references=expected_responses_texto, rouge_types=["rougeL"])
print(f"\nROUGE-L F1 Score: {rouge_result['rougeL']:.4f}")

# BERT-F1
bert_result = bert_score.compute(
    predictions=predictions_text,
    references=expected_responses_texto,
    lang="en",
    device="cuda"
)
print(f" BERTScore F1 (average): {sum(bert_result['f1']) /
↳len(bert_result['f1']):.4f}")

```

ROUGE-L F1 Score: 0.3879

tokenizer_config.json: 0%| | 0.00/25.0 [00:00<?, ?B/s]

config.json: 0%| | 0.00/482 [00:00<?, ?B/s]

vocab.json: 0%| | 0.00/899k [00:00<?, ?B/s]

merges.txt: 0%| | 0.00/456k [00:00<?, ?B/s]

tokenizer.json: 0%| | 0.00/1.36M [00:00<?, ?B/s]

model.safetensors: 0%| | 0.00/1.42G [00:00<?, ?B/s]

BERTScore F1 (average): 0.8849

- BERTScore F1 (Semantic Similarity (even if different words are used)): 0.89
- ROUGE-L: (Literal Overlap (exact words or phrases)) = 0.39
 - Compared to the run of Llama v2 it's getting a doubled better Rouge and a little better BERTScore.

11 Q3 - Transfer learnt model built on a pretrained LLM such as GPT-2 or TinyLlama using fine-tuning mechanisms such as PEFT (LoRA, QLoRA) etc

```

[3]: import pandas as pd
from IPython.display import display

# Define the data

```

```

data = {
    "Model": ["Transfer learnt model built on a pretrained LLM"] * 7,
    "Tasks and Comments": [
        "Preprocessing Steps - 1. Tokenization",
        "Training - Model built, train and test handled correctly?",
        "Evaluation - ROUGE-L Score, BERT Score?",
        "Interpretation using Lime",
        "1st round of tuning - What was the issue faced/tuned?",
        "2nd round of tuning - What was the issue faced/tuned?",
        "Final AUC value? Next steps Recommended?"
    ],
    "Status": ["Done", "Done", "Done", "Done", "Pending", "Pending", "Pending"] ,
    "Individual Responsible": ["Jiisy", "Jissy", "Jissy", "Jissy", "", "", ""]
}

# Create DataFrame
df = pd.DataFrame(data)

# Display the table
display(df)

```

	Model \	Tasks and Comments	Status \
0	Transfer learnt model built on a pretrained LLM		
1	Transfer learnt model built on a pretrained LLM		
2	Transfer learnt model built on a pretrained LLM		
3	Transfer learnt model built on a pretrained LLM		
4	Transfer learnt model built on a pretrained LLM		
5	Transfer learnt model built on a pretrained LLM		
6	Transfer learnt model built on a pretrained LLM		
0		Preprocessing Steps - 1. Tokenization	Done
1		Training - Model built, train and test handled...	Done
2		Evaluation - ROUGE-L Score, BERT Score?	Done
3		Interpretation using Lime	Done
4		1st round of tuning - What was the issue faced...	Pending
5		2nd round of tuning - What was the issue faced...	Pending
6		Final AUC value? Next steps Recommended?	Pending
	Individual Responsible		
0	Jiisy		
1	Jissy		
2	Jissy		
3	Jissy		
4			
5			
6			

```
[ ]: # Step 1: Install Necessary Libraries
```

```
!pip install transformers  
!pip install peft  
!pip install datasets
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.11/dist-packages (4.50.2)  
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from transformers) (3.18.0)  
Requirement already satisfied: huggingface-hub<1.0,>=0.26.0 in /usr/local/lib/python3.11/dist-packages (from transformers) (0.29.3)  
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.11/dist-packages (from transformers) (2.0.2)  
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from transformers) (24.2)  
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.11/dist-packages (from transformers) (6.0.2)  
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.11/dist-packages (from transformers) (2024.11.6)  
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from transformers) (2.32.3)  
Requirement already satisfied: tokenizers<0.22,>=0.21 in /usr/local/lib/python3.11/dist-packages (from transformers) (0.21.1)  
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.11/dist-packages (from transformers) (0.5.3)  
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.11/dist-packages (from transformers) (4.67.1)  
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.11/dist-packages (from huggingface-hub<1.0,>=0.26.0->transformers) (2025.3.0)  
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.11/dist-packages (from huggingface-hub<1.0,>=0.26.0->transformers) (4.13.0)  
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests->transformers) (3.4.1)  
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests->transformers) (3.10)  
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests->transformers) (2.3.0)  
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests->transformers) (2025.1.31)  
Requirement already satisfied: peft in /usr/local/lib/python3.11/dist-packages (0.14.0)  
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.11/dist-packages (from peft) (2.0.2)  
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from peft) (24.2)
```

Requirement already satisfied: psutil in /usr/local/lib/python3.11/dist-packages (from peft) (5.9.5)

Requirement already satisfied: pyyaml in /usr/local/lib/python3.11/dist-packages (from peft) (6.0.2)

Requirement already satisfied: torch>=1.13.0 in /usr/local/lib/python3.11/dist-packages (from peft) (2.6.0+cu124)

Requirement already satisfied: transformers in /usr/local/lib/python3.11/dist-packages (from peft) (4.50.2)

Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from peft) (4.67.1)

Requirement already satisfied: accelerate>=0.21.0 in /usr/local/lib/python3.11/dist-packages (from peft) (1.5.2)

Requirement already satisfied: safetensors in /usr/local/lib/python3.11/dist-packages (from peft) (0.5.3)

Requirement already satisfied: huggingface-hub>=0.25.0 in /usr/local/lib/python3.11/dist-packages (from peft) (0.29.3)

Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from huggingface-hub>=0.25.0->peft) (3.18.0)

Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.11/dist-packages (from huggingface-hub>=0.25.0->peft) (2025.3.0)

Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from huggingface-hub>=0.25.0->peft) (2.32.3)

Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.11/dist-packages (from huggingface-hub>=0.25.0->peft) (4.13.0)

Requirement already satisfied: networkx in /usr/local/lib/python3.11/dist-packages (from torch>=1.13.0->peft) (3.4.2)

Requirement already satisfied: Jinja2 in /usr/local/lib/python3.11/dist-packages (from torch>=1.13.0->peft) (3.1.6)

Collecting nvidia-cuda-nvrtc-cu12==12.4.127 (from torch>=1.13.0->peft)

 Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)

Collecting nvidia-cuda-runtime-cu12==12.4.127 (from torch>=1.13.0->peft)

 Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)

Collecting nvidia-cuda-cupti-cu12==12.4.127 (from torch>=1.13.0->peft)

 Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)

Collecting nvidia-cudnn-cu12==9.1.0.70 (from torch>=1.13.0->peft)

 Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)

Collecting nvidia-cublas-cu12==12.4.5.8 (from torch>=1.13.0->peft)

 Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)

Collecting nvidia-cufft-cu12==11.2.1.3 (from torch>=1.13.0->peft)

 Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)

```

Collecting nvidia-curand-cu12==10.3.5.147 (from torch>=1.13.0->peft)
  Downloading nvidia_curand_cu12-10.3.5.147-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cusolver-cu12==11.6.1.9 (from torch>=1.13.0->peft)
  Downloading nvidia_cusolver_cu12-11.6.1.9-py3-none-
manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cusparselt-cu12==0.6.2 (from torch>=1.13.0->peft)
  Downloading nvidia_cusparselt_cu12-0.6.2-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Requirement already satisfied: nvidia-cusparse-cu12==12.3.1.170 in
/usr/local/lib/python3.11/dist-packages (from torch>=1.13.0->peft) (12.3.1.170)
Requirement already satisfied: nvidia-nccl-cu12==2.21.5 in
/usr/local/lib/python3.11/dist-packages (from torch>=1.13.0->peft) (2.21.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.4.127 in
/usr/local/lib/python3.11/dist-packages (from torch>=1.13.0->peft) (12.4.127)
Collecting nvidia-nvjitlink-cu12==12.4.127 (from torch>=1.13.0->peft)
  Downloading nvidia_nvjitlink_cu12-12.4.127-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Requirement already satisfied: triton==3.2.0 in /usr/local/lib/python3.11/dist-
packages (from torch>=1.13.0->peft) (3.2.0)
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.11/dist-
packages (from torch>=1.13.0->peft) (1.13.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.11/dist-packages (from
sympy==1.13.1->torch>=1.13.0->peft) (1.3.0)
Requirement already satisfied: regex!=2019.12.17 in
/usr/local/lib/python3.11/dist-packages (from transformers->peft) (2024.11.6)
Requirement already satisfied: tokenizers<0.22,>=0.21 in
/usr/local/lib/python3.11/dist-packages (from transformers->peft) (0.21.1)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.11/dist-packages (from jinja2->torch>=1.13.0->peft)
(3.0.2)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.11/dist-packages (from requests->huggingface-
hub>=0.25.0->peft) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-
packages (from requests->huggingface-hub>=0.25.0->peft) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.11/dist-packages (from requests->huggingface-
hub>=0.25.0->peft) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.11/dist-packages (from requests->huggingface-
hub>=0.25.0->peft) (2025.1.31)
Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-manylinux2014_x86_64.whl (363.4
MB)
363.4/363.4 MB
4.2 MB/s eta 0:00:00
Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-

```

```

manylinux2014_x86_64.whl (13.8 MB)
13.8/13.8 MB
76.9 MB/s eta 0:00:00
Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-
manylinux2014_x86_64.whl (24.6 MB)
24.6/24.6 MB
62.7 MB/s eta 0:00:00
Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-
manylinux2014_x86_64.whl (883 kB)
883.7/883.7 kB
39.8 MB/s eta 0:00:00
Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl
(664.8 MB)
664.8/664.8 MB
1.6 MB/s eta 0:00:00
Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-manylinux2014_x86_64.whl
(211.5 MB)
211.5/211.5 MB
4.5 MB/s eta 0:00:00
Downloading nvidia_curand_cu12-10.3.5.147-py3-none-
manylinux2014_x86_64.whl (56.3 MB)
56.3/56.3 MB
11.0 MB/s eta 0:00:00
Downloading nvidia_cusolver_cu12-11.6.1.9-py3-none-
manylinux2014_x86_64.whl (127.9 MB)
127.9/127.9 MB
6.9 MB/s eta 0:00:00
Downloading nvidia_cusparses_cu12-12.3.1.170-py3-none-
manylinux2014_x86_64.whl (207.5 MB)
207.5/207.5 MB
4.9 MB/s eta 0:00:00
Downloading nvidia_nvjitlink_cu12-12.4.127-py3-none-
manylinux2014_x86_64.whl (21.1 MB)
21.1/21.1 MB
58.9 MB/s eta 0:00:00
Installing collected packages: nvidia-nvjitlink-cu12, nvidia-curand-cu12,
nvidia-cufft-cu12, nvidia-cuda-runtime-cu12, nvidia-cuda-nvrtc-cu12, nvidia-
cuda-cupti-cu12, nvidia-cublas-cu12, nvidia-cusparses-cu12, nvidia-cudnn-cu12,
nvidia-cusolver-cu12
  Attempting uninstall: nvidia-nvjitlink-cu12
    Found existing installation: nvidia-nvjitlink-cu12 12.5.82
    Uninstalling nvidia-nvjitlink-cu12-12.5.82:
      Successfully uninstalled nvidia-nvjitlink-cu12-12.5.82
  Attempting uninstall: nvidia-curand-cu12
    Found existing installation: nvidia-curand-cu12 10.3.6.82
    Uninstalling nvidia-curand-cu12-10.3.6.82:
      Successfully uninstalled nvidia-curand-cu12-10.3.6.82
  Attempting uninstall: nvidia-cufft-cu12

```

```

Found existing installation: nvidia-cufft-cu12 11.2.3.61
Uninstalling nvidia-cufft-cu12-11.2.3.61:
  Successfully uninstalled nvidia-cufft-cu12-11.2.3.61
Attempting uninstall: nvidia-cuda-runtime-cu12
Found existing installation: nvidia-cuda-runtime-cu12 12.5.82
Uninstalling nvidia-cuda-runtime-cu12-12.5.82:
  Successfully uninstalled nvidia-cuda-runtime-cu12-12.5.82
Attempting uninstall: nvidia-cuda-nvrtc-cu12
Found existing installation: nvidia-cuda-nvrtc-cu12 12.5.82
Uninstalling nvidia-cuda-nvrtc-cu12-12.5.82:
  Successfully uninstalled nvidia-cuda-nvrtc-cu12-12.5.82
Attempting uninstall: nvidia-cuda-cupti-cu12
Found existing installation: nvidia-cuda-cupti-cu12 12.5.82
Uninstalling nvidia-cuda-cupti-cu12-12.5.82:
  Successfully uninstalled nvidia-cuda-cupti-cu12-12.5.82
Attempting uninstall: nvidia-cublas-cu12
Found existing installation: nvidia-cublas-cu12 12.5.3.2
Uninstalling nvidia-cublas-cu12-12.5.3.2:
  Successfully uninstalled nvidia-cublas-cu12-12.5.3.2
Attempting uninstall: nvidia-cusparse-cu12
Found existing installation: nvidia-cusparse-cu12 12.5.1.3
Uninstalling nvidia-cusparse-cu12-12.5.1.3:
  Successfully uninstalled nvidia-cusparse-cu12-12.5.1.3
Attempting uninstall: nvidia-cudnn-cu12
Found existing installation: nvidia-cudnn-cu12 9.3.0.75
Uninstalling nvidia-cudnn-cu12-9.3.0.75:
  Successfully uninstalled nvidia-cudnn-cu12-9.3.0.75
Attempting uninstall: nvidia-cusolver-cu12
Found existing installation: nvidia-cusolver-cu12 11.6.3.83
Uninstalling nvidia-cusolver-cu12-11.6.3.83:
  Successfully uninstalled nvidia-cusolver-cu12-11.6.3.83
Successfully installed nvidia-cublas-cu12-12.4.5.8 nvidia-cuda-cupti-
cu12-12.4.127 nvidia-cuda-nvrtc-cu12-12.4.127 nvidia-cuda-runtime-cu12-12.4.127
nvidia-cudnn-cu12-9.1.0.70 nvidia-cufft-cu12-11.2.1.3 nvidia-curand-
cu12-10.3.5.147 nvidia-cusolver-cu12-11.6.1.9 nvidia-cusparse-cu12-12.3.1.170
nvidia-nvjitlink-cu12-12.4.127
Collecting datasets
  Downloading datasets-3.5.0-py3-none-any.whl.metadata (19 kB)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-
packages (from datasets) (3.18.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.11/dist-
packages (from datasets) (2.0.2)
Requirement already satisfied: pyarrow>=15.0.0 in
/usr/local/lib/python3.11/dist-packages (from datasets) (18.1.0)
Collecting dill<0.3.9,>=0.3.0 (from datasets)
  Downloading dill-0.3.8-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages
(from datasets) (2.2.2)

```

Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.11/dist-packages (from datasets) (2.32.3)

Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.11/dist-packages (from datasets) (4.67.1)

Collecting xxhash (from datasets)

 Downloading xxhash-3.5.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (12 kB)

Collecting multiprocessing<0.70.17 (from datasets)

 Downloading multiprocessing-0.70.16-py311-none-any.whl.metadata (7.2 kB)

Collecting fsspec<=2024.12.0,>=2023.1.0 (from fsspec[http]<=2024.12.0,>=2023.1.0->datasets)

 Downloading fsspec-2024.12.0-py3-none-any.whl.metadata (11 kB)

Requirement already satisfied: aiohttp in /usr/local/lib/python3.11/dist-packages (from datasets) (3.11.14)

Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.11/dist-packages (from datasets) (0.29.3)

Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from datasets) (24.2)

Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.11/dist-packages (from datasets) (6.0.2)

Requirement already satisfied: aiohappyeyeballs>=2.3.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (2.6.1)

Requirement already satisfied: aiosignal>=1.1.2 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (1.3.2)

Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (25.3.0)

Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (1.5.0)

Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (6.2.0)

Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (0.3.1)

Requirement already satisfied: yarll<2.0,>=1.17.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (1.18.3)

Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.11/dist-packages (from huggingface-hub>=0.24.0->datasets) (4.13.0)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests>=2.32.2->datasets) (3.4.1)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests>=2.32.2->datasets) (3.10)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests>=2.32.2->datasets) (2.3.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests>=2.32.2->datasets)


```

(2025.1.31)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.11/dist-packages (from pandas->datasets) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-
packages (from pandas->datasets) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-
packages (from pandas->datasets) (2025.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-
packages (from python-dateutil>=2.8.2->pandas->datasets) (1.17.0)
Downloading datasets-3.5.0-py3-none-any.whl (491 kB)
      491.2/491.2 kB
11.9 MB/s eta 0:00:00
Downloading dill-0.3.8-py3-none-any.whl (116 kB)
      116.3/116.3 kB
12.1 MB/s eta 0:00:00
Downloading fsspec-2024.12.0-py3-none-any.whl (183 kB)
      183.9/183.9 kB
19.2 MB/s eta 0:00:00
Downloading multiprocessing-0.70.16-py311-none-any.whl (143 kB)
      143.5/143.5 kB
15.5 MB/s eta 0:00:00
Downloading
xxhash-3.5.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (194 kB)
      194.8/194.8 kB
20.2 MB/s eta 0:00:00
Installing collected packages: xxhash, fsspec, dill, multiprocessing,
datasets
  Attempting uninstall: fsspec
    Found existing installation: fsspec 2025.3.0
    Uninstalling fsspec-2025.3.0:
      Successfully uninstalled fsspec-2025.3.0
ERROR: pip's dependency resolver does not currently take into account all
the packages that are installed. This behaviour is the source of the following
dependency conflicts.
gcsfs 2025.3.0 requires fsspec==2025.3.0, but you have fsspec 2024.12.0 which is
incompatible.

Successfully installed datasets-3.5.0 dill-0.3.8 fsspec-2024.12.0
multiprocessing-0.70.16 xxhash-3.5.0

```

12 Import Libraries

```
[ ]: # Step 2: Import Libraries and Load Dataset
import pandas as pd
from datasets import Dataset
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig, get_peft_model
```

13 Load Data

```
[ ]: # Step 3: Load train.csv and valid.csv (use your own file paths here or upload
      them)
train_file_path = '/content/train.csv'
valid_file_path = '/content/valid.csv'
```

```
[ ]: train_df = pd.read_csv(train_file_path)
      valid_df = pd.read_csv(valid_file_path)
```

```
[ ]: # Display first few rows of the datasets to inspect
      train_df.head(), valid_df.head()
```

```
[ ]: (
      answers \
0 Kids who are bipolar, in their manic stages, v...
1 Equifax, transunion and experian.
2 Women eat at least 1,200 calories daily and me...
3 Because Caffeine increases the stress hormone ...
4 Kent County

      query \
0 why do children get aggressive
1 which credit bureau is used the most for auto ...
2 what is the minimum healthy calorie intake
3 why is coffee making gain weight
4 what county is grand rapids, mi in

      finalpassage
0 At the same time, despite claiming the review ...
1 Best Answer: both of those answers are wrong. ...
2 Safe Intakes. If you're not supervised by a me...
3 Is coffee making you fat? If you are overweigh...
4 Located in Grand Rapids, Michigan, the 61st Di... ,

      answers \
0 It is a very popular first name for men and al...
1 No worries
2 Electronic medical recordElectronic Medical Re...
3 It is the skin around its neck.
4 On the inner surface of your arm near your elbow.
```

```

                                query \
0          how popular is the name conrad
1          disney hakuna matata meaning
2          what does emr stand for
3          what is a dog's ruff
4  what is the part on your arm where they draw b...

                                finalpassage
0  The name Conrad is a baby boy name. The name C...
1  Hakuna Matata (song) Hakuna Matata is a song f...
2  The EMR (electronic medical record) is used to...
3  No problem! Many dogs, like many people, are j...
4  One, called venipuncture, involves drawing a v... )

```

```
[ ]: # Step 4: Convert pandas DataFrame to HuggingFace dataset format
train_data = Dataset.from_pandas(train_df)
valid_data = Dataset.from_pandas(valid_df)
```

```
[ ]: from transformers import AutoTokenizer, AutoModelForCausalLM
import torch
```

14 Load Model

```
[ ]: # Step 7: Load distilgpt2 Model
model_name = "distilgpt2"
# Load tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_name)

# If there is no padding token, set the EOS token as padding
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

def tokenize_function(examples):
    # Combine query and passage to form input text
    input_texts = [
        (q if q is not None else "") + " " + (p if p is not None else "")
        for q, p in zip(examples["query"], examples["finalpassage"])
    ]

    tokenized_output = tokenizer(input_texts, padding="max_length",
    ↪truncation=True, max_length=512)

    # Causal LM expects labels to be the same as input_ids
    tokenized_output["labels"] = tokenized_output["input_ids"].copy()
```

```

    return tokenized_output

# Apply tokenization on both datasets
train_data = train_data.map(tokenize_function, batched=True, batch_size=8,
    ↪keep_in_memory=True)
valid_data = valid_data.map(tokenize_function, batched=True, batch_size=8,
    ↪keep_in_memory=True)

# Step 8: Load Model and Move to Device
from transformers import AutoModelForCausalLM

# Load the model
model = AutoModelForCausalLM.from_pretrained(model_name)

# Set device to GPU if available, else fallback to CPU
device = "cuda" if torch.cuda.is_available() else "cpu"

# Move the model to the appropriate device
model.to(device)

```

```

/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94:
UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab
(https://huggingface.co/settings/tokens), set it as secret in your Google Colab
and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access
public models or datasets.

```

```

    warnings.warn(

tokenizer_config.json:  0%|          | 0.00/26.0 [00:00<?, ?B/s]
config.json:          0%|          | 0.00/762 [00:00<?, ?B/s]
vocab.json:           0%|          | 0.00/1.04M [00:00<?, ?B/s]
merges.txt:           0%|          | 0.00/456k [00:00<?, ?B/s]
tokenizer.json:       0%|          | 0.00/1.36M [00:00<?, ?B/s]
Map:                  0%|          | 0/120000 [00:00<?, ? examples/s]
Map:                  0%|          | 0/10585 [00:00<?, ? examples/s]
model.safetensors:    0%|          | 0.00/353M [00:00<?, ?B/s]
generation_config.json: 0%|          | 0.00/124 [00:00<?, ?B/s]

```

```

[ ]: GPT2LMHeadModel(
      (transformer): GPT2Model(
        (wte): Embedding(50257, 768)

```

```

(wpe): Embedding(1024, 768)
(drop): Dropout(p=0.1, inplace=False)
(h): ModuleList(
  (0-5): 6 x GPT2Block(
    (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
    (attn): GPT2Attention(
      (c_attn): Conv1D(nf=2304, nx=768)
      (c_proj): Conv1D(nf=768, nx=768)
      (attn_dropout): Dropout(p=0.1, inplace=False)
      (resid_dropout): Dropout(p=0.1, inplace=False)
    )
    (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
    (mlp): GPT2MLP(
      (c_fc): Conv1D(nf=3072, nx=768)
      (c_proj): Conv1D(nf=768, nx=3072)
      (act): NewGELUActivation()
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
)
(ln_f): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
)
(lm_head): Linear(in_features=768, out_features=50257, bias=False)
)

```

```

[ ]: # Step 6: Sampling the data (taking random samples for testing)
train_sample = train_data.shuffle(seed=42).select([i for i in list(range(5))]) ␣
    ↪ # First 5 samples
valid_sample = valid_data.shuffle(seed=42).select([i for i in list(range(5))]) ␣
    ↪ # First 5 samples

# Inspect sample
train_sample, valid_sample

```

```

[ ]: (Dataset({
  features: ['answers', 'query', 'finalpassage', 'input_ids',
'attention_mask', 'labels'],
  num_rows: 5
}),
Dataset({
  features: ['answers', 'query', 'finalpassage', 'input_ids',
'attention_mask', 'labels'],
  num_rows: 5
}))

```

15 Configure LoRA and Apply to the Model

```
[ ]: from transformers import AutoModelForCausalLM
import torch

# Step 7: Load distilgpt2 Model
model_name = "distilgpt2"
model = AutoModelForCausalLM.from_pretrained(model_name)

# Step 8: Configure LoRA and Apply to the Model
lora_config = LoraConfig(
    r=8, # Rank of low-rank matrices
    lora_alpha=32, # Scaling factor
    lora_dropout=0.05, # Dropout rate for LoRA layers
    bias="none", # No bias in LoRA
    task_type="CAUSAL_LM" # Causal language modeling task
)

# Apply LoRA to the model
model = get_peft_model(model, lora_config)

# Check if a GPU is available, otherwise use CPU
device = "cuda" if torch.cuda.is_available() else "cpu"

# Move the model to the selected device (either GPU or CPU)
model.to(device)
```

```
/usr/local/lib/python3.11/dist-packages/peft/tuners/lora/layer.py:1264:
UserWarning: fan_in_fan_out is set to False but the target module is `Conv1D`.
Setting fan_in_fan_out to True.
warnings.warn(
```

```
[ ]: PeftModelForCausalLM(
  (base_model): LoraModel(
    (model): GPT2LMHeadModel(
      (transformer): GPT2Model(
        (wte): Embedding(50257, 768)
        (wpe): Embedding(1024, 768)
        (drop): Dropout(p=0.1, inplace=False)
        (h): ModuleList(
          (0-5): 6 x GPT2Block(
            (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
            (attn): GPT2Attention(
              (c_attn): lora.Linear(
                (base_layer): Conv1D(nf=2304, nx=768)
                (lora_dropout): ModuleDict(
                  (default): Dropout(p=0.05, inplace=False)
                )
              )
            )
          )
        )
      )
    )
```

```

        (lora_A): ModuleDict(
          (default): Linear(in_features=768, out_features=8, bias=False)
        )
        (lora_B): ModuleDict(
          (default): Linear(in_features=8, out_features=2304,
bias=False)
        )
        (lora_embedding_A): ParameterDict()
        (lora_embedding_B): ParameterDict()
        (lora_magnitude_vector): ModuleDict()
      )
      (c_proj): Conv1D(nf=768, nx=768)
      (attn_dropout): Dropout(p=0.1, inplace=False)
      (resid_dropout): Dropout(p=0.1, inplace=False)
    )
    (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
    (mlp): GPT2MLP(
      (c_fc): Conv1D(nf=3072, nx=768)
      (c_proj): Conv1D(nf=768, nx=3072)
      (act): NewGELUActivation()
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
)
(ln_f): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
)
(lm_head): Linear(in_features=768, out_features=50257, bias=False)
)
)
)

```

16 Generate Text using model

```

[ ]: # Step 9: Generate Text Using the LoRA Model
def generate_text(prompt, max_length=50):
    # Tokenize input prompt
    inputs = tokenizer(prompt, return_tensors="pt").to(device)

    # Generate text
    outputs = model.generate(
        inputs['input_ids'],
        max_length=max_length,
        num_return_sequences=1,
        no_repeat_ngram_size=2, # Prevent repeating n-grams
        temperature=0.7, # Sampling temperature
        top_k=50, # Top-k sampling
    )

```

```

        top_p=0.95, # Top-p sampling
    )

    # Decode and return generated text
    generated_text = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return generated_text

# Step 10: Test Generation on a Sample
sample_prompt = "Once upon a time in a land far away"
generated_sample = generate_text(sample_prompt)
print(generated_sample)

```

```

/usr/local/lib/python3.11/dist-
packages/transformers/generation/configuration_utils.py:628: UserWarning:
`do_sample` is set to `False`. However, `temperature` is set to `0.7` -- this
flag is only used in sample-based generation modes. You should set
`do_sample=True` or unset `temperature`.
  warnings.warn(
/usr/local/lib/python3.11/dist-
packages/transformers/generation/configuration_utils.py:633: UserWarning:
`do_sample` is set to `False`. However, `top_p` is set to `0.95` -- this flag is
only used in sample-based generation modes. You should set `do_sample=True` or
unset `top_p`.
  warnings.warn(
The attention mask and the pad token id were not set. As a consequence, you may
observe unexpected behavior. Please pass your input's `attention_mask` to obtain
reliable results.
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
The attention mask is not set and cannot be inferred from input because pad
token is same as eos token. As a consequence, you may observe unexpected
behavior. Please pass your input's `attention_mask` to obtain reliable results.

Once upon a time in a land far away, the sun is shining.

```

The sun rises and the moon rises. The sun sets. It is the night of the day.

```

[ ]: # Example of selecting a random 10% sample of the dataset
train_data_sampled = train_data.shuffle(seed=42).select([i for i in
↪range(int(len(train_data) * 0.1))])
valid_data_sampled = valid_data.shuffle(seed=42).select([i for i in
↪range(int(len(valid_data) * 0.1))])

```


17 Fine Tuning and train the model

```
[ ]: # Step 11: Fine-tune the model (Optional) - Fine-tuning on the dataset can be
      ↪ done here

from transformers import Trainer, TrainingArguments

# Define training arguments
training_args = TrainingArguments(
    output_dir='./results',          # output directory
    num_train_epochs=3,              # number of training epochs
    per_device_train_batch_size=8,   # batch size for training
    per_device_eval_batch_size=8,    # batch size for evaluation
    logging_dir='./logs',            # directory for storing logs
)

# Use sampled datasets for training
trainer = Trainer(
    model=model,                    # the model to train
    args=training_args,              # training arguments
    train_dataset=train_data_sampled, # sampled training dataset
    eval_dataset=valid_data_sampled,  # sampled validation dataset
)

# Train the model
trainer.train()
```

No label_names provided for model class `PeftModelForCausalLM`. Since `PeftModel` hides base models input arguments, if label_names is not given, label_names can't be set automatically within `Trainer`. Note that empty label_names list will be used instead.

wandb: WARNING The `run\_name` is currently set to the same value as `TrainingArguments.output\_dir`. If this was not intended, please specify a different run name by setting the `TrainingArguments.run\_name` parameter.

wandb: Using wandb-core as the SDK backend. Please refer to <https://wandb.me/wandb-core> for more information.

<IPython.core.display.Javascript object>

wandb: Logging into wandb.ai. (Learn how to deploy a W&B server locally: <https://wandb.me/wandb-server>)

wandb: You can find your API key in your browser here: <https://wandb.ai/authorize>

wandb: Paste an API key from your profile and hit enter:

.....

wandb: WARNING If you're specifying your api key in code, ensure this code is not shared publicly.

wandb: WARNING Consider setting the WANDB_API_KEY environment variable, or running `wandb login` from the command line.

wandb: No netrc file found, creating one.

wandb: Appending key for api.wandb.ai to your netrc file:
/root/.netrc

wandb: Currently logged in as: jissyj1996
(jissyj1996-loyalist-college) to <https://api.wandb.ai>. Use
`wandb login --relogin` to force relogin

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

`loss\_type=None` was set in the config but it is unrecognised.Using the default
loss: `ForCausalLMLoss`.

<IPython.core.display.HTML object>

```
[ ]: TrainOutput(global_step=4500, training_loss=1.3867266167534722,  
metrics={'train_runtime': 2574.7034, 'train_samples_per_second': 13.982,  
'train_steps_per_second': 1.748, 'total_flos': 4719648964608000.0, 'train_loss':  
1.3867266167534722, 'epoch': 3.0})
```

```
[ ]: !pip install evaluate
```

Collecting evaluate

Downloading evaluate-0.4.3-py3-none-any.whl.metadata (9.2 kB)

Requirement already satisfied: datasets>=2.0.0 in

/usr/local/lib/python3.11/dist-packages (from evaluate) (3.5.0)

Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.11/dist-
packages (from evaluate) (2.0.2)

Requirement already satisfied: dill in /usr/local/lib/python3.11/dist-packages
(from evaluate) (0.3.8)

Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages
(from evaluate) (2.2.2)

Requirement already satisfied: requests>=2.19.0 in
/usr/local/lib/python3.11/dist-packages (from evaluate) (2.32.3)

Requirement already satisfied: tqdm>=4.62.1 in /usr/local/lib/python3.11/dist-
packages (from evaluate) (4.67.1)

Requirement already satisfied: xxhash in /usr/local/lib/python3.11/dist-packages
(from evaluate) (3.5.0)

Requirement already satisfied: multiprocessing in /usr/local/lib/python3.11/dist-
packages (from evaluate) (0.70.16)

Requirement already satisfied: fsspec>=2021.05.0 in
/usr/local/lib/python3.11/dist-packages (from fsspec[http]>=2021.05.0->evaluate)
(2024.12.0)

Requirement already satisfied: huggingface-hub>=0.7.0 in
/usr/local/lib/python3.11/dist-packages (from evaluate) (0.29.3)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-
packages (from evaluate) (24.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-
packages (from datasets>=2.0.0->evaluate) (3.18.0)
Requirement already satisfied: pyarrow>=15.0.0 in
/usr/local/lib/python3.11/dist-packages (from datasets>=2.0.0->evaluate)
(18.1.0)
Requirement already satisfied: aiohttp in /usr/local/lib/python3.11/dist-
packages (from datasets>=2.0.0->evaluate) (3.11.14)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.11/dist-
packages (from datasets>=2.0.0->evaluate) (6.0.2)
Requirement already satisfied: typing-extensions>=3.7.4.3 in
/usr/local/lib/python3.11/dist-packages (from huggingface-hub>=0.7.0->evaluate)
(4.13.0)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->evaluate)
(3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-
packages (from requests>=2.19.0->evaluate) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->evaluate)
(2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->evaluate)
(2025.1.31)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.11/dist-packages (from pandas->evaluate) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-
packages (from pandas->evaluate) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-
packages (from pandas->evaluate) (2025.2)
Requirement already satisfied: aiohappyeyeballs>=2.3.0 in
/usr/local/lib/python3.11/dist-packages (from
aiohttp->datasets>=2.0.0->evaluate) (2.6.1)
Requirement already satisfied: aiosignal>=1.1.2 in
/usr/local/lib/python3.11/dist-packages (from
aiohttp->datasets>=2.0.0->evaluate) (1.3.2)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.11/dist-
packages (from aiohttp->datasets>=2.0.0->evaluate) (25.3.0)
Requirement already satisfied: frozenlist>=1.1.1 in
/usr/local/lib/python3.11/dist-packages (from
aiohttp->datasets>=2.0.0->evaluate) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in
/usr/local/lib/python3.11/dist-packages (from
aiohttp->datasets>=2.0.0->evaluate) (6.2.0)
Requirement already satisfied: propcache>=0.2.0 in

```

/usr/local/lib/python3.11/dist-packages (from
aiohttp>datasets>=2.0.0->evaluate) (0.3.1)
Requirement already satisfied: yarll<2.0,>=1.17.0 in
/usr/local/lib/python3.11/dist-packages (from
aiohttp>datasets>=2.0.0->evaluate) (1.18.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-
packages (from python-dateutil>=2.8.2->pandas->evaluate) (1.17.0)
Downloading evaluate-0.4.3-py3-none-any.whl (84 kB)
84.0/84.0 kB
2.4 MB/s eta 0:00:00
Installing collected packages: evaluate
Successfully installed evaluate-0.4.3

```

```
[ ]: !pip install rouge_score
```

```

Collecting rouge_score
  Downloading rouge_score-0.1.2.tar.gz (17 kB)
  Preparing metadata (setup.py) ... done
Requirement already satisfied: absl-py in /usr/local/lib/python3.11/dist-
packages (from rouge_score) (1.4.0)
Requirement already satisfied: nltk in /usr/local/lib/python3.11/dist-packages
(from rouge_score) (3.9.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages
(from rouge_score) (2.0.2)
Requirement already satisfied: six>=1.14.0 in /usr/local/lib/python3.11/dist-
packages (from rouge_score) (1.17.0)
Requirement already satisfied: click in /usr/local/lib/python3.11/dist-packages
(from nltk->rouge_score) (8.1.8)
Requirement already satisfied: joblib in /usr/local/lib/python3.11/dist-packages
(from nltk->rouge_score) (1.4.2)
Requirement already satisfied: regex>=2021.8.3 in
/usr/local/lib/python3.11/dist-packages (from nltk->rouge_score) (2024.11.6)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages
(from nltk->rouge_score) (4.67.1)
Building wheels for collected packages: rouge_score
  Building wheel for rouge_score (setup.py) ... done
  Created wheel for rouge_score: filename=rouge_score-0.1.2-py3-none-any.whl
size=24934
sha256=75f278d0fbfaab56220cfe545b48e2f161ead4c4b00b508392698d5dbed0678c
  Stored in directory: /root/.cache/pip/wheels/1e/19/43/8a442dc83660ca25e163e1bd
1f89919284ab0d0c1475475148
Successfully built rouge_score
Installing collected packages: rouge_score
Successfully installed rouge_score-0.1.2

```

```
[ ]: !pip install bert_score
```

```

Collecting bert_score
  Downloading bert_score-0.3.13-py3-none-any.whl.metadata (15 kB)

```

Requirement already satisfied: torch>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from bert_score) (2.6.0+cu124)

Requirement already satisfied: pandas>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from bert_score) (2.2.2)

Requirement already satisfied: transformers>=3.0.0 in /usr/local/lib/python3.11/dist-packages (from bert_score) (4.50.2)

Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from bert_score) (2.0.2)

Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from bert_score) (2.32.3)

Requirement already satisfied: tqdm>=4.31.1 in /usr/local/lib/python3.11/dist-packages (from bert_score) (4.67.1)

Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-packages (from bert_score) (3.10.0)

Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.11/dist-packages (from bert_score) (24.2)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.0.1->bert_score) (2.8.2)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.0.1->bert_score) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.0.1->bert_score) (2025.2)

Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score) (3.18.0)

Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score) (4.13.0)

Requirement already satisfied: networkx in /usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score) (3.4.2)

Requirement already satisfied: jinja2 in /usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score) (3.1.6)

Requirement already satisfied: fsspec in /usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score) (2024.12.0)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score) (12.4.127)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score) (12.4.127)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score) (12.4.127)

Requirement already satisfied: nvidia-cudnn-cu12==9.1.0.70 in /usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score) (9.1.0.70)

Requirement already satisfied: nvidia-cublas-cu12==12.4.5.8 in /usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score) (12.4.5.8)

Requirement already satisfied: nvidia-cufft-cu12==11.2.1.3 in

```

/usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score)
(11.2.1.3)
Requirement already satisfied: nvidia-curand-cu12==10.3.5.147 in
/usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score)
(10.3.5.147)
Requirement already satisfied: nvidia-cusolver-cu12==11.6.1.9 in
/usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score)
(11.6.1.9)
Requirement already satisfied: nvidia-cuspars-cu12==12.3.1.170 in
/usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score)
(12.3.1.170)
Requirement already satisfied: nvidia-cusparselt-cu12==0.6.2 in
/usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score) (0.6.2)
Requirement already satisfied: nvidia-nccl-cu12==2.21.5 in
/usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score) (2.21.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.4.127 in
/usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score)
(12.4.127)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.4.127 in
/usr/local/lib/python3.11/dist-packages (from torch>=1.0.0->bert_score)
(12.4.127)
Requirement already satisfied: triton==3.2.0 in /usr/local/lib/python3.11/dist-
packages (from torch>=1.0.0->bert_score) (3.2.0)
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.11/dist-
packages (from torch>=1.0.0->bert_score) (1.13.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.11/dist-packages (from
sympy==1.13.1->torch>=1.0.0->bert_score) (1.3.0)
Requirement already satisfied: huggingface-hub<1.0,>=0.26.0 in
/usr/local/lib/python3.11/dist-packages (from transformers>=3.0.0->bert_score)
(0.29.3)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.11/dist-
packages (from transformers>=3.0.0->bert_score) (6.0.2)
Requirement already satisfied: regex!=2019.12.17 in
/usr/local/lib/python3.11/dist-packages (from transformers>=3.0.0->bert_score)
(2024.11.6)
Requirement already satisfied: tokenizers<0.22,>=0.21 in
/usr/local/lib/python3.11/dist-packages (from transformers>=3.0.0->bert_score)
(0.21.1)
Requirement already satisfied: safetensors>=0.4.3 in
/usr/local/lib/python3.11/dist-packages (from transformers>=3.0.0->bert_score)
(0.5.3)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.11/dist-packages (from matplotlib->bert_score) (1.3.1)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-
packages (from matplotlib->bert_score) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.11/dist-packages (from matplotlib->bert_score) (4.56.0)

```

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->bert_score) (1.4.8)

Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/dist-packages (from matplotlib->bert_score) (11.1.0)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->bert_score) (3.2.3)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests->bert_score) (3.4.1)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests->bert_score) (3.10)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests->bert_score) (2.3.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests->bert_score) (2025.1.31)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas>=1.0.1->bert_score) (1.17.0)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dist-packages (from jinja2->torch>=1.0.0->bert_score) (3.0.2)

Downloading bert_score-0.3.13-py3-none-any.whl (61 kB)

61.1/61.1 kB

3.6 MB/s eta 0:00:00

Installing collected packages: bert_score

Successfully installed bert_score-0.3.13

18 Evaluation using ROUGE and BERTScore metrics

```
[ ]: import evaluate

# Load ROUGE and BERTScore metrics
rouge = evaluate.load("rouge")
bertscore = evaluate.load("bertscore")

# Function to evaluate the model performance using ROUGE-L and BERT-F1
def evaluate_model(model, tokenizer, dataset):
    predictions = []
    references = []

    for example in dataset:
        input_text = example["query"]
        target_text = example["finalpassage"]

        # Generate response from the model
        inputs = tokenizer(input_text, return_tensors="pt").to(device)
        generated_ids = model.generate(inputs['input_ids'], max_length=150)
        generated_text = tokenizer.decode(generated_ids[0],
        ↪skip_special_tokens=True)
```

```

        predictions.append(generated_text)
        references.append(target_text)

    # Compute ROUGE-L and BERT-F1
    rouge_result = rouge.compute(predictions=predictions, references=references)
    bertscore_result = bertscore.compute(predictions=predictions,
    ↪references=references, model_type="bert-base-uncased")

    return rouge_result, bertscore_result

# Evaluate the fine-tuned model on the validation set
rouge_result, bertscore_result = evaluate_model(model, tokenizer, valid_sample)

# Output evaluation results
print(f"ROUGE-L: {rouge_result}")
print(f"BERTScore F1: {bertscore_result['f1']}")

```

The attention mask and the pad token id were not set. As a consequence, you may observe unexpected behavior. Please pass your input's `attention\_mask` to obtain reliable results.

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

The attention mask and the pad token id were not set. As a consequence, you may observe unexpected behavior. Please pass your input's `attention\_mask` to obtain reliable results.

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

The attention mask and the pad token id were not set. As a consequence, you may observe unexpected behavior. Please pass your input's `attention\_mask` to obtain reliable results.

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

The attention mask and the pad token id were not set. As a consequence, you may observe unexpected behavior. Please pass your input's `attention\_mask` to obtain reliable results.

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

The attention mask and the pad token id were not set. As a consequence, you may observe unexpected behavior. Please pass your input's `attention\_mask` to obtain reliable results.

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

tokenizer_config.json: 0%| | 0.00/48.0 [00:00<?, ?B/s]

config.json: 0%| | 0.00/570 [00:00<?, ?B/s]

vocab.txt: 0%| | 0.00/232k [00:00<?, ?B/s]

tokenizer.json: 0%| | 0.00/466k [00:00<?, ?B/s]

model.safetensors: 0%| | 0.00/440M [00:00<?, ?B/s]

ROUGE-L: {'rouge1': np.float64(0.14647387141305052), 'rouge2':
np.float64(0.024852071005917162), 'rougeL': np.float64(0.12429599903963232),


```
'rougeLsum': np.float64(0.12429599903963232)}
BERTScore F1: [0.4410865604877472, 0.48622220754623413, 0.3779492974281311,
0.5065504908561707, 0.5807270407676697]
```

```
[ ]: def generate_text(prompt, max_length=50):
    # Tokenize input prompt and create attention mask
    inputs = tokenizer(prompt, return_tensors="pt", padding=True,
    ↪truncation=True, max_length=max_length, return_attention_mask=True).
    ↪to(device)

    # Generate text
    outputs = model.generate(
        inputs['input_ids'],
        attention_mask=inputs['attention_mask'], # Pass the attention mask here
        max_length=max_length,
        num_return_sequences=1,
        no_repeat_ngram_size=2, # Prevent repeating n-grams
        temperature=0.7, # Sampling temperature
        top_k=50, # Top-k sampling
        top_p=0.95, # Top-p sampling
    )

    # Decode and return generated text
    generated_text = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return generated_text
```

19 Save Model

```
[ ]: model.save_pretrained("/content/fine_tuned_model")
tokenizer.save_pretrained("/content/fine_tuned_model")
```

```
[ ]: ('/content/fine_tuned_model/tokenizer_config.json',
      '/content/fine_tuned_model/special_tokens_map.json',
      '/content/fine_tuned_model/vocab.json',
      '/content/fine_tuned_model/merges.txt',
      '/content/fine_tuned_model/added_tokens.json',
      '/content/fine_tuned_model/tokenizer.json')
```

```
[ ]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[ ]: model.save_pretrained("/content/drive/MyDrive/fine_tuned_model")
tokenizer.save_pretrained("/content/drive/MyDrive/fine_tuned_model")
```

```
[ ]: ('/content/drive/MyDrive/fine_tuned_model/tokenizer_config.json',
      '/content/drive/MyDrive/fine_tuned_model/special_tokens_map.json',
      '/content/drive/MyDrive/fine_tuned_model/vocab.json',
      '/content/drive/MyDrive/fine_tuned_model/merges.txt',
      '/content/drive/MyDrive/fine_tuned_model/added_tokens.json',
      '/content/drive/MyDrive/fine_tuned_model/tokenizer.json')
```

20 Lime

```
[ ]: from lime.lime_text import LimeTextExplainer
import numpy as np
import torch

# Ensure tokenizer has a padding token
if tokenizer.pad_token is None:
    tokenizer.add_special_tokens({'pad_token': '[PAD]'})
    model.resize_token_embeddings(len(tokenizer))

# Prediction function for LIME
def predict_proba(texts, batch_size=8):
    probs_list = []

    for i in range(0, len(texts), batch_size):
        batch = texts[i : i + batch_size]

        # Tokenize input text with padding & truncation
        inputs = tokenizer(batch, return_tensors="pt", padding=True,
        ↪truncation=True, max_length=50).to(model.device)

        with torch.no_grad():
            outputs = model(**inputs)
            logits = outputs.logits[:, -1, :] # Extract last token logits

        probs = torch.nn.functional.softmax(logits, dim=-1).cpu().numpy()
        probs_list.append(probs)

    return np.vstack(probs_list)

# Initialize LIME explainer
explainer = LimeTextExplainer(class_names=["Negative", "Positive"]) # Adjust
    ↪based on your model

# Sample text for explanation
sample_prompt = "The service at this restaurant was absolutely amazing!"

# Generate explanation with optimized sample size
```

```
exp = explainer.explain_instance(sample_prompt, predict_proba, num_features=10, num_samples=100)
```

```
# Display explanation  
exp.show_in_notebook()
```

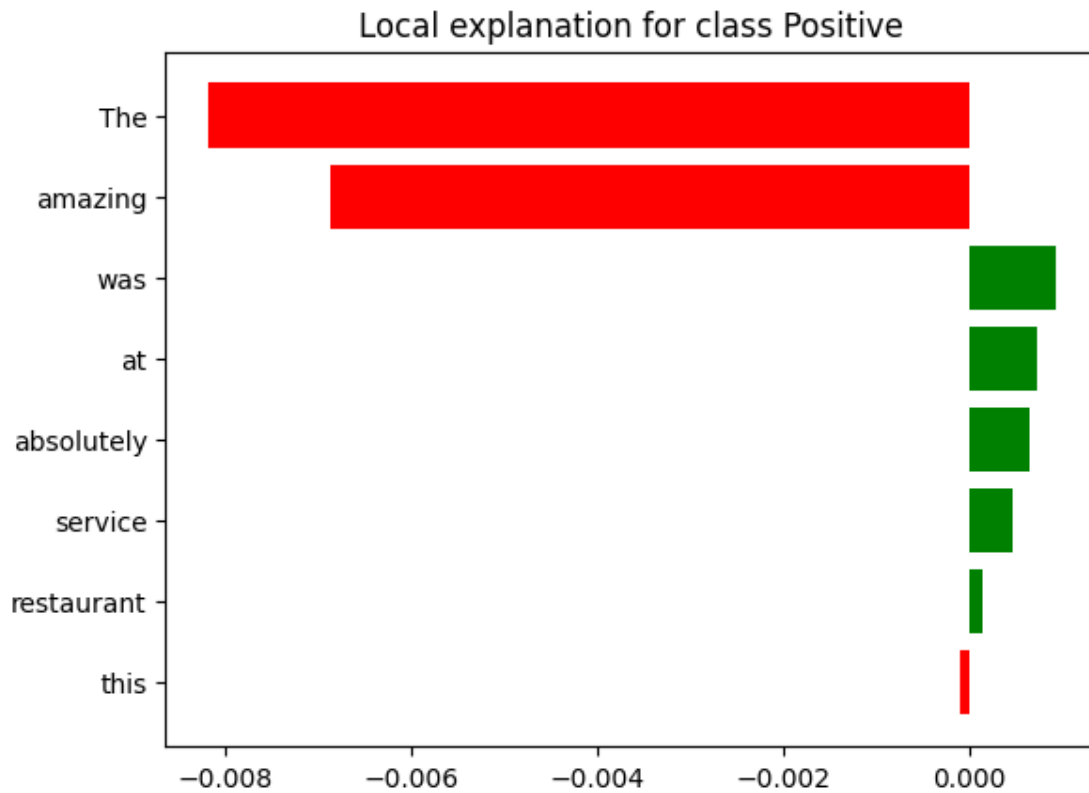
<IPython.core.display.HTML object>

```
[ ]: import matplotlib.pyplot as plt  
  
# Show explanation in Colab  
exp.save_to_file('lime_explanation.html') # Save LIME output as an HTML file  
from google.colab import files  
files.download('lime_explanation.html') # Download file for local viewing
```

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

```
[ ]: import matplotlib.pyplot as plt  
  
# Generate the explanation for the given prompt  
exp = explainer.explain_instance(sample_prompt, predict_proba, num_features=10)  
  
# Display the explanation as a matplotlib figure  
exp.as_pyplot_figure()  
plt.show()
```



20.1 Q1. Causal Transformer based models using pretrained word embeddings and position embeddings

```
[8]: import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
from transformers import AutoTokenizer
import pandas as pd
import numpy as np
import lime.lime_text
import gensim.downloader as api
```

```
[9]: # Load the dataset
train_path = "train.csv"
test_path = "valid.csv"

# Attempt different encodings if UTF-8 fails
encodings = ["utf-8", "ISO-8859-1", "latin1"]

for enc in encodings:
```

```

try:
    train_df = pd.read_csv(train_path, encoding=enc)
    test_df = pd.read_csv(test_path, encoding=enc)
    print(f"Successfully loaded with encoding: {enc}")
    break # Stop if successful
except UnicodeDecodeError:
    print(f"Encoding {enc} failed, trying next...")

```

Successfully loaded with encoding: utf-8

```

[10]: print(train_df.shape)
      train_df.head()

```

(120000, 3)

```

[10]:                                     answers \
0 Kids who are bipolar, in their manic stages, v...
1 Equifax, transunion and experian.
2 Women eat at least 1,200 calories daily and me...
3 Because Caffeine increases the stress hormone ...
4 Kent County

                                     query \
0 why do children get aggressive
1 which credit bureau is used the most for auto ...
2 what is the minimum healthy calorie intake
3 why is coffee making gain weight
4 what county is grand rapids, mi in

                                     finalpassage
0 At the same time, despite claiming the review ...
1 Best Answer: both of those answers are wrong. ...
2 Safe Intakes. If you're not supervised by a me...
3 Is coffee making you fat? If you are overweigh...
4 Located in Grand Rapids, Michigan, the 61st Di...

```

```

[11]: print(test_df.shape)
      test_df.head()

```

(10585, 3)

```

[11]:                                     answers \
0 It is a very popular first name for men and al...
1 No worries
2 Electronic medical recordElectronic Medical Re...
3 It is the skin around its neck.
4 On the inner surface of your arm near your elbow.

                                     query \

```

```

0             how popular is the name conrad
1             disney hakuna matata meaning
2             what does emr stand for
3             what is a dog's ruff
4  what is the part on your arm where they draw b...

```

finalpassage

```

0  The name Conrad is a baby boy name. The name C...
1  Hakuna Matata (song) Hakuna Matata is a song f...
2  The EMR (electronic medical record) is used to...
3  No problem! Many dogs, like many people, are j...
4  One, called venipuncture, involves drawing a v...

```

```

[12]: print(train_df.isnull().sum())
      train_df.dropna(inplace=True)
      print(train_df.isnull().sum())

```

```

answers      55
query         0
finalpassage 24
dtype: int64
answers      0
query         0
finalpassage 0
dtype: int64

```

```

[13]: print(test_df.isnull().sum())
      test_df.dropna(inplace=True)
      print(test_df.isnull().sum())

```

```

answers      4
query         0
finalpassage 1
dtype: int64
answers      0
query         0
finalpassage 0
dtype: int64

```

```

[14]: one_row = train_df.iloc[0]

      print("Query: ", one_row["query"])
      print("Answer: ", one_row["answers"])
      print("Final Passage: ", one_row["finalpassage"])

```

Query: why do children get aggressive

Answer: Kids who are bipolar, in their manic stages, very frequently become aggressive. They lose self-control, they become impulsive. On the other end of the spectrum, when they become depressed, although aggression is less common,

they can become irritable, and sometimes that irritability and cantankerousness causes kids to lash out.

Final Passage: At the same time, despite claiming the review demonstrates a link between playing violent video games and aggression, the authors acknowledged that some studies were inconsistent and that present research is insufficient to establish whether this can lead to criminal violence or delinquency. Kids who are bipolar, in their manic stages, very frequently become aggressive. They lose self-control, they become impulsive. On the other end of the spectrum, when they become depressed, although aggression is less common, they can become irritable, and sometimes that irritability and cantankerousness causes kids to lash out.

```
[15]: train_df.drop("finalpassage", axis=1, inplace = True)
```

```
[16]: test_df.drop("finalpassage", axis=1, inplace = True)
```

```
[17]: import re
def remove_special_chars(text):
    return re.sub(r'[^a-zA-Z0-9\s.,?!]', '', text)

train_df["query"] = train_df["query"].apply(remove_special_chars)
train_df["answers"] = train_df["answers"].apply(remove_special_chars)

test_df["query"] = test_df["query"].apply(remove_special_chars)
test_df["answers"] = test_df["answers"].apply(remove_special_chars)
```

```
[18]: def remove_emojis_and_mentions(text):
    # Remove mentions, hashtags, and URLs
    text = re.sub(r'@[\w]+', ' ', text) # Remove mentions
    text = re.sub(r'http\S+|www\S+', ' ', text) # Remove URLs
    text = re.sub(r'[\x00-\x7F]+', '', text) # Remove emojis
    return text

train_df["query"] = train_df["query"].apply(remove_emojis_and_mentions)
train_df["answers"] = train_df["answers"].apply(remove_emojis_and_mentions)

test_df["query"] = test_df["query"].apply(remove_emojis_and_mentions)
test_df["answers"] = test_df["answers"].apply(remove_emojis_and_mentions)
```

```
[19]: train_df["query"] = train_df["query"].str.lower()
train_df["answers"] = train_df["answers"].str.lower()

test_df["query"] = test_df["query"].str.lower()
test_df["answers"] = test_df["answers"].str.lower()
```

```
[20]: # Load Word2Vec pretrained embeddings
word2vec = api.load("word2vec-google-news-300")
embed_dim = word2vec.vector_size
```

[-----] 1.4% 23.4/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 5.1% 84.0/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 8.9% 147.9/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 12.6% 210.3/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:

NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)

NotebookApp.rate_limit_window=3.0 (secs)

[=====-----] 16.5% 275.0/1662.8MB
downloaded

IOPub message rate exceeded.

The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.

To change this limit, set the config variable

`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:

NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)

NotebookApp.rate_limit_window=3.0 (secs)

[=====-----] 20.2% 336.2/1662.8MB
downloaded

IOPub message rate exceeded.

The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.

To change this limit, set the config variable

`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:

NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)

NotebookApp.rate_limit_window=3.0 (secs)

[=====-----] 24.0% 398.8/1662.8MB
downloaded

IOPub message rate exceeded.

The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.

To change this limit, set the config variable

`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:

NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)

NotebookApp.rate_limit_window=3.0 (secs)

[=====-----] 27.8% 461.7/1662.8MB
downloaded

IOPub message rate exceeded.

The notebook server will temporarily stop sending output

to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====-----] 31.3% 521.2/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====-----] 34.1% 566.8/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====-----] 35.3% 586.5/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 37.9% 630.0/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 39.2% 651.1/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 41.8% 695.3/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 43.0% 714.4/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:

NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)

NotebookApp.rate_limit_window=3.0 (secs)

[=====] 45.5% 756.4/1662.8MB
downloaded

IOPub message rate exceeded.

The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.

To change this limit, set the config variable

`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:

NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)

NotebookApp.rate_limit_window=3.0 (secs)

[=====] 46.6% 774.5/1662.8MB
downloaded

IOPub message rate exceeded.

The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.

To change this limit, set the config variable

`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:

NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)

NotebookApp.rate_limit_window=3.0 (secs)

[=====] 49.2% 818.0/1662.8MB
downloaded

IOPub message rate exceeded.

The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.

To change this limit, set the config variable

`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:

NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)

NotebookApp.rate_limit_window=3.0 (secs)

[=====] 50.4% 838.1/1662.8MB
downloaded

IOPub message rate exceeded.

The notebook server will temporarily stop sending output

to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 52.9% 879.9/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 54.2% 901.2/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 56.8% 943.8/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 58.0% 965.2/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 60.5% 1006.7/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 61.9% 1028.6/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 64.5% 1072.4/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:

NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)

NotebookApp.rate_limit_window=3.0 (secs)

[=====] 65.7% 1091.7/1662.8MB
downloaded

IOPub message rate exceeded.

The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.

To change this limit, set the config variable

`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:

NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)

NotebookApp.rate_limit_window=3.0 (secs)

[=====] 68.2% 1133.4/1662.8MB
downloaded

IOPub message rate exceeded.

The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.

To change this limit, set the config variable

`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:

NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)

NotebookApp.rate_limit_window=3.0 (secs)

[=====] 69.5% 1155.1/1662.8MB
downloaded

IOPub message rate exceeded.

The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.

To change this limit, set the config variable

`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:

NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)

NotebookApp.rate_limit_window=3.0 (secs)

[=====] 72.0% 1197.4/1662.8MB
downloaded

IOPub message rate exceeded.

The notebook server will temporarily stop sending output

to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 73.4% 1219.8/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 75.7% 1259.1/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 77.0% 1280.2/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 79.5% 1321.2/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 80.8% 1343.9/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 83.4% 1386.5/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 84.8% 1409.6/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:

NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)

NotebookApp.rate_limit_window=3.0 (secs)

[=====] 87.2% 1450.7/1662.8MB
downloaded

IOPub message rate exceeded.

The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.

To change this limit, set the config variable

`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:

NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)

NotebookApp.rate_limit_window=3.0 (secs)

[=====] 88.7% 1474.7/1662.8MB
downloaded

IOPub message rate exceeded.

The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.

To change this limit, set the config variable

`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:

NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)

NotebookApp.rate_limit_window=3.0 (secs)

[=====] 91.0% 1513.6/1662.8MB
downloaded

IOPub message rate exceeded.

The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.

To change this limit, set the config variable

`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:

NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)

NotebookApp.rate_limit_window=3.0 (secs)

[=====] 92.5% 1537.7/1662.8MB
downloaded

IOPub message rate exceeded.

The notebook server will temporarily stop sending output

to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 94.8% 1576.0/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 96.1% 1598.5/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

[=====] 98.6% 1639.9/1662.8MB
downloaded

IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub\_msg\_rate\_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)

```
[=====] 100.0% 1662.8/1662.8MB  
downloaded
```

```
[22]: word2vec.save("word2vec-google-news-300.model")
```

```
[23]: # Tokenizer Setup  
model_name = "bert-base-uncased"  
tokenizer = AutoTokenizer.from_pretrained(model_name)  
max_len = 128  
  
train_df["query_tokens"] = train_df["query"].apply(lambda x: tokenizer.  
    ↪ encode_plus(x, max_length=max_len, padding="max_length", truncation=True,  
    ↪ return_tensors='pt'))  
train_df["answer_tokens"] = train_df["answers"].apply(lambda x: tokenizer.  
    ↪ encode_plus(x, max_length=max_len, padding="max_length", truncation=True,  
    ↪ return_tensors='pt'))  
test_df["query_tokens"] = test_df["query"].apply(lambda x: tokenizer.  
    ↪ encode_plus(x, max_length=max_len, padding="max_length", truncation=True,  
    ↪ return_tensors='pt'))  
test_df["answer_tokens"] = test_df["answers"].apply(lambda x: tokenizer.  
    ↪ encode_plus(x, max_length=max_len, padding="max_length", truncation=True,  
    ↪ return_tensors='pt'))
```

```
tokenizer_config.json: 0%|          | 0.00/48.0 [00:00<?, ?B/s]
```

```
config.json: 0%|          | 0.00/570 [00:00<?, ?B/s]
```

```
vocab.txt: 0%|          | 0.00/232k [00:00<?, ?B/s]
```

```
tokenizer.json: 0%|          | 0.00/466k [00:00<?, ?B/s]
```

```
[24]: def tokens_to_word2vec(tokens):  
    vectors = [word2vec[word] if word in word2vec else np.zeros(embed_dim) for  
    ↪ word in tokens]  
    return np.array(vectors, dtype=np.float32)  
  
# Convert train tokens to embeddings  
train_df["query_embeddings"] = train_df["query_tokens"].  
    ↪ apply(tokens_to_word2vec)  
train_df["answer_embeddings"] = train_df["answer_tokens"].  
    ↪ apply(tokens_to_word2vec)  
  
# Convert test tokens to embeddings  
test_df["query_embeddings"] = test_df["query_tokens"].apply(tokens_to_word2vec)  
test_df["answer_embeddings"] = test_df["answer_tokens"].  
    ↪ apply(tokens_to_word2vec)
```

```
[25]: class ChatDataset(Dataset):  
    def __init__(self, queries, answers):
```

```

        self.queries = queries
        self.answers = answers

    def __len__(self):
        return len(self.queries)

    def __getitem__(self, idx):
        return torch.tensor(self.queries[idx]), torch.tensor(self.answers[idx])

```

```

[26]: # Sample
train_subset = train_df.sample(frac=0.7, random_state=42).
↳reset_index(drop=True) # 70% of train_df

# Prepare Data
train_dataset = ChatDataset(train_subset["query_embeddings"].tolist(),
↳train_df["answer_embeddings"].tolist())
test_dataset = ChatDataset(test_df["query_embeddings"].tolist(),
↳test_df["answer_embeddings"].tolist())

#Load Data
train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=8, shuffle=False)

```

```

[27]: from tqdm import tqdm

# --- Model ---
class CausalTransformer(nn.Module):
    def __init__(self, d_model=300, nhead=6, num_layers=6, dim_feedforward=512,
↳max_len=128):
        super().__init__()
        self.pos_embedding = nn.Embedding(max_len, d_model)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=nhead,
            dim_feedforward=dim_feedforward,
            activation='gelu'
        )
        self.encoder = nn.TransformerEncoder(encoder_layer,
↳num_layers=num_layers)

        decoder_layer = nn.TransformerDecoderLayer(
            d_model=d_model,
            nhead=nhead,
            dim_feedforward=dim_feedforward,
            activation='gelu'
        )

```

```

        self.decoder = nn.TransformerDecoder(decoder_layer,
↪num_layers=num_layers)

        self.fc_out = nn.Linear(d_model, d_model)

    def forward(self, x, pos_ids):
        x = x + self.pos_embedding(pos_ids)

        seq_len = x.size(1)
        mask = torch.triu(torch.ones(seq_len, seq_len), diagonal=1).bool().to(x.
↪device)

        x = x.transpose(0, 1)  # (seq_len, batch, d_model)

        memory = self.encoder(x, mask=mask)
        output = self.decoder(x, memory, tgt_mask=mask, memory_mask=mask)
        output = output.transpose(0, 1)  # (batch, seq_len, d_model)

        return self.fc_out(output)

```

```

[28]: import torch
import torch.optim as optim
import torch.nn as nn
import matplotlib.pyplot as plt
import json
from tqdm import tqdm

# Assuming the CausalTransformer model and train_loader are already defined

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
model = CausalTransformer().to(device)

criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=1e-4)
num_epochs = 2

# List to store the loss per batch
batch_losses = []

for epoch in range(num_epochs):
    model.train()
    total_loss = 0.0

    for batch_idx, (query_emb, answer_emb) in enumerate(tqdm(train_loader,
↪desc=f"Epoch {epoch+1}")):
        query_emb = query_emb.to(device)

```

```

        answer_emb = answer_emb.to(device)

        batch_size, seq_len, _ = query_emb.size()
        pos_ids = torch.arange(seq_len).unsqueeze(0).repeat(batch_size, 1).
        ↪to(device)

        output = model(query_emb, pos_ids)

        if output.size(1) != answer_emb.size(1):
            output = output[:, :answer_emb.size(1), :]

        loss = criterion(output, answer_emb)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        batch_losses.append(loss.item()) # Store loss per batch

        # Optionally print loss every 10000 batches
        if batch_idx % 10000 == 0:
            print(f"[Batch {batch_idx}] Loss: {loss.item():.4f}")

    avg_epoch_loss = total_loss / len(train_loader)
    print(f"Epoch {epoch+1}/{num_epochs} - Loss: {avg_epoch_loss:.4f}")

```

/opt/anaconda3/lib/python3.11/site-packages/torch/nn/modules/transformer.py:379:
 UserWarning: enable_nested_tensor is True, but self.use_nested_tensor is False
 because encoder_layer.self_attn.batch_first was not True(use batch_first for
 better inference performance)

warnings.warn(

Using device: cpu

Epoch 1: 0%| | 3/10494 [00:00<15:03, 11.62it/s]

[Batch 0] Loss: 0.3446

Epoch 1: 95%| | 10003/10494 [09:44<00:27, 17.54it/s]

[Batch 10000] Loss: 0.0000

Epoch 1: 100%| | 10494/10494 [10:11<00:00, 17.17it/s]

Epoch 1/2 - Loss: 0.0010

Epoch 2: 0%| | 2/10494 [00:00<09:21, 18.68it/s]

[Batch 0] Loss: 0.0000

Epoch 2: 95%| | 10005/10494 [09:10<00:24, 20.22it/s]

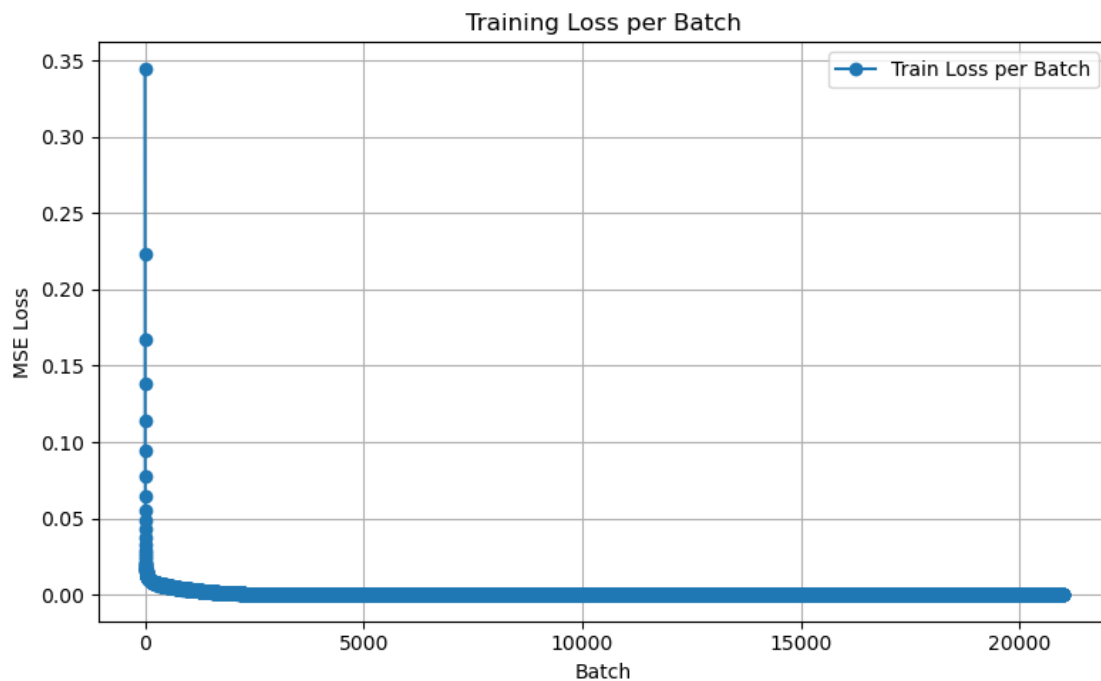
[Batch 10000] Loss: 0.0000

Epoch 2: 100% | 10494/10494 [09:36<00:00, 18.22it/s]

Epoch 2/2 - Loss: 0.0000

```
[29]: # Save batch losses to a file (JSON)
with open('batch_losses.json', 'w') as f:
    json.dump(batch_losses, f)

# Plotting the training loss per batch
plt.figure(figsize=(8, 5))
plt.plot(batch_losses, marker='o', label="Train Loss per Batch")
plt.title("Training Loss per Batch")
plt.xlabel("Batch")
plt.ylabel("MSE Loss")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```



```
[30]: # Save full model (architecture + weights)
torch.save(model, "causal_transformer_full_2.pth")
model
```



```

[30]: CausalTransformer(
  (pos_embedding): Embedding(128, 300)
  (encoder): TransformerEncoder(
    (layers): ModuleList(
      (0-5): 6 x TransformerEncoderLayer(
        (self_attn): MultiheadAttention(
          (out_proj): NonDynamicallyQuantizableLinear(in_features=300,
out_features=300, bias=True)
        )
        (linear1): Linear(in_features=300, out_features=512, bias=True)
        (dropout): Dropout(p=0.1, inplace=False)
        (linear2): Linear(in_features=512, out_features=300, bias=True)
        (norm1): LayerNorm((300,), eps=1e-05, elementwise_affine=True)
        (norm2): LayerNorm((300,), eps=1e-05, elementwise_affine=True)
        (dropout1): Dropout(p=0.1, inplace=False)
        (dropout2): Dropout(p=0.1, inplace=False)
      )
    )
  )
  (decoder): TransformerDecoder(
    (layers): ModuleList(
      (0-5): 6 x TransformerDecoderLayer(
        (self_attn): MultiheadAttention(
          (out_proj): NonDynamicallyQuantizableLinear(in_features=300,
out_features=300, bias=True)
        )
        (multihead_attn): MultiheadAttention(
          (out_proj): NonDynamicallyQuantizableLinear(in_features=300,
out_features=300, bias=True)
        )
        (linear1): Linear(in_features=300, out_features=512, bias=True)
        (dropout): Dropout(p=0.1, inplace=False)
        (linear2): Linear(in_features=512, out_features=300, bias=True)
        (norm1): LayerNorm((300,), eps=1e-05, elementwise_affine=True)
        (norm2): LayerNorm((300,), eps=1e-05, elementwise_affine=True)
        (norm3): LayerNorm((300,), eps=1e-05, elementwise_affine=True)
        (dropout1): Dropout(p=0.1, inplace=False)
        (dropout2): Dropout(p=0.1, inplace=False)
        (dropout3): Dropout(p=0.1, inplace=False)
      )
    )
  )
  (fc_out): Linear(in_features=300, out_features=300, bias=True)
)

```

```

[31]: import torch
import numpy as np

```

```

from sklearn.metrics.pairwise import cosine_similarity
import matplotlib.pyplot as plt

def evaluate_cosine_similarity(model, test_loader, device, subset_size=5000):
    model.eval()
    predictions, references = [], []

    with torch.no_grad():
        # Iterate through the test set (subset_size)
        for batch_idx, (query, answer) in enumerate(test_loader):
            if batch_idx * test_loader.batch_size >= subset_size:
                break

            query = query.to(device).float()
            answer = answer.to(device).float()
            pos_ids = torch.zeros(query.size(0), query.size(1), dtype=torch.
↪long).to(device)

            # Get model output (generated answer embeddings)
            output = model(query, pos_ids)

            # Store generated answer embeddings (predictions) and ground truth
↪answer embeddings
            for i in range(output.size(0)):
                pred_emb = output[i].cpu().numpy() # Generated answer
↪embedding (predicted by model)
                ref_emb = answer[i].cpu().numpy() # Ground truth answer
↪embedding

                # Average the embeddings to represent the entire sequence
                pred_avg = pred_emb.mean(axis=0)
                ref_avg = ref_emb.mean(axis=0)

                predictions.append(pred_avg) # Store predicted embeddings
                references.append(ref_avg) # Store ground-truth embeddings

    # Convert lists to numpy arrays for similarity computation
    pred_embeddings = np.array(predictions)
    ref_embeddings = np.array(references)

    # Compute cosine similarity between predicted and reference embeddings
    similarities = cosine_similarity(pred_embeddings, ref_embeddings)

    # Calculate the average cosine similarity across the evaluated subset
    avg_cos_sim = np.mean(similarities)

    # Plot similarity values for each test sample

```

```

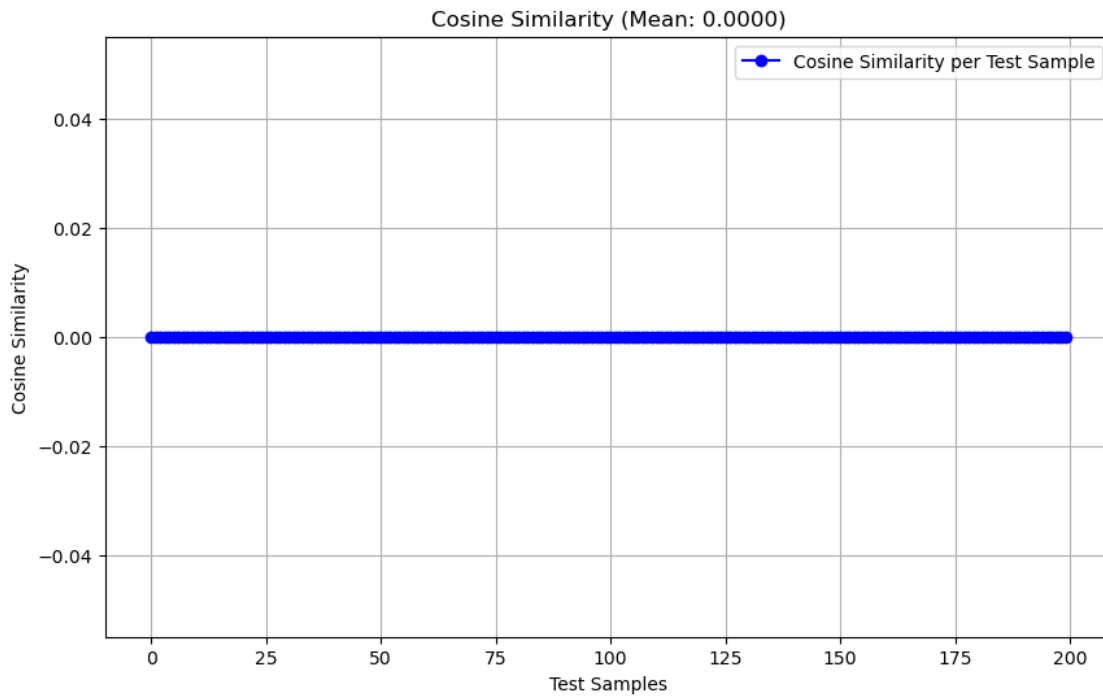
plt.figure(figsize=(10, 6))
plt.plot(np.diag(similarities), label="Cosine Similarity per Test Sample",
color='b', marker='o')
plt.title(f"Cosine Similarity (Mean: {avg_cos_sim:.4f})")
plt.xlabel('Test Samples')
plt.ylabel('Cosine Similarity')
plt.grid(True)
plt.legend()
plt.show()

return avg_cos_sim

# Example: Running the evaluation function on a subset of the test data (200
samples)
avg_cos_sim = evaluate_cosine_similarity(model, test_loader, device,
subset_size=200)

print(f"Average Cosine Similarity: {avg_cos_sim:.4f}")

```



Average Cosine Similarity: 0.0000

```

[48]: import torch
from lime.lime_text import LimeTextExplainer
from transformers import AutoTokenizer

```

```

from transformers import AutoModel
import numpy as np

# Assuming model and tokenizer are already loaded
model = torch.load("causal_transformer_full_2.pth")
model.eval()
model.to(device)

tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')

# Load pretrained BERT model to extract embeddings
bert_model = AutoModel.from_pretrained("bert-base-uncased")
bert_model.to(device)
bert_model.eval()

projection = torch.nn.Linear(768, 300).to(device)

def predict_proba(texts):
    tokenized = tokenizer(
        texts,
        padding=True,
        truncation=True,
        max_length=max_len,
        return_tensors='pt'
    ).to(device)

    input_ids = tokenized['input_ids']
    attention_mask = tokenized['attention_mask']

    with torch.no_grad():
        bert_outputs = bert_model(input_ids=input_ids,
        ↪attention_mask=attention_mask)
        embeddings = bert_outputs.last_hidden_state
        embeddings = projection(embeddings)

    # Create pos_ids: [0, 1, ..., seq_len-1]
    seq_len = embeddings.shape[1]
    pos_ids = torch.arange(seq_len).unsqueeze(0).expand(embeddings.size(0), -1).
    ↪to(device)

    with torch.no_grad():
        out = model(embeddings, pos_ids)
        final_output = out[:, -1, :] # Last token's output

    probs = torch.softmax(final_output, dim=-1).cpu().numpy()
    return probs
# Instantiate a LimeTextExplainer

```

```

explainer = LimeTextExplainer() # Adjust class names to your dataset

# Example text to explain
sample_text = ["The restaurant was amazing!"]

# Use LIME to explain the prediction
explanation = explainer.explain_instance(
    sample_text[0], # Example text to explain
    predict_proba, # Prediction function
    num_features=10 # Number of features to show in the explanation
)

# Visualize the explanation
explanation.show_in_notebook()

```

```

/var/folders/tz/_kr6c8m90dl2p9klr693qf6w0000gn/T/ipykernel_47991/2600984048.py:7
: FutureWarning: You are using `torch.load` with `weights_only=False` (the
current default value), which uses the default pickle module implicitly. It is
possible to construct malicious pickle data which will execute arbitrary code
during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for
more details). In a future release, the default value for `weights_only` will be
flipped to `True`. This limits the functions that could be executed during
unpickling. Arbitrary objects will no longer be allowed to be loaded via this
mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting
`weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this
experimental feature.
    model = torch.load("causal_transformer_full_2.pth")

<IPython.core.display.HTML object>

```

20.2 Tuning

```

[49]: import torch
import torch.nn as nn

class CausalTransformer(nn.Module):
    def __init__(
        self,
        d_model=300,
        nhead=6,
        num_layers=6,
        dim_feedforward=512,
        dropout=0.1,
        max_len=128,
    ):

```

```

super().__init__()

self.pos_embedding = nn.Embedding(max_len, d_model)
self.dropout = nn.Dropout(p=dropout)

encoder_layer = nn.TransformerEncoderLayer(
    d_model=d_model,
    nhead=nhead,
    dim_feedforward=dim_feedforward,
    dropout=dropout,
    activation='gelu'
)
self.encoder = nn.TransformerEncoder(encoder_layer,
↪num_layers=num_layers)

decoder_layer = nn.TransformerDecoderLayer(
    d_model=d_model,
    nhead=nhead,
    dim_feedforward=dim_feedforward,
    dropout=dropout,
    activation='gelu'
)
self.decoder = nn.TransformerDecoder(decoder_layer,
↪num_layers=num_layers)

self.fc_out = nn.Linear(d_model, d_model)

def forward(self, x, pos_ids):
    x = self.dropout(x + self.pos_embedding(pos_ids))

    seq_len = x.size(1)
    mask = torch.triu(torch.ones(seq_len, seq_len), diagonal=1).bool().to(x.
↪device)

    x = x.transpose(0, 1) # (seq_len, batch, d_model)

    memory = self.encoder(x, mask=mask)
    output = self.decoder(x, memory, tgt_mask=mask, memory_mask=mask)
    output = output.transpose(0, 1) # (batch, seq_len, d_model)

    return self.fc_out(output)

```

```

[50]: import torch
import torch.nn as nn
import torch.optim as optim
from tqdm import tqdm
import matplotlib.pyplot as plt

```

```

# -- Training Loop Function --
def train_model(model, train_loader, num_epochs=2, lr=1e-4):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Using device: {device}")
    model = model.to(device)

    criterion = nn.MSELoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    batch_losses = []

    for epoch in range(num_epochs):
        model.train()
        total_loss = 0.0

        loop = tqdm(train_loader, desc=f"Epoch {epoch+1}/{num_epochs}",
            ↪leave=False)
        for batch_idx, (query_emb, answer_emb) in enumerate(loop):
            query_emb = query_emb.to(device)
            answer_emb = answer_emb.to(device)

            batch_size, seq_len, _ = query_emb.size()
            pos_ids = torch.arange(seq_len).unsqueeze(0).repeat(batch_size, 1).
            ↪to(device)

            output = model(query_emb, pos_ids)

            if output.size(1) != answer_emb.size(1):
                output = output[:, :answer_emb.size(1), :]

            loss = criterion(output, answer_emb)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total_loss += loss.item()
            batch_losses.append(loss.item())

            if batch_idx % 1000 == 0:
                loop.set_postfix(loss=f"{loss.item():.4f}")

        avg_epoch_loss = total_loss / len(train_loader)
        print(f" Epoch {epoch+1} finished - Avg Loss: {avg_epoch_loss:.4f}")

    return batch_losses

```

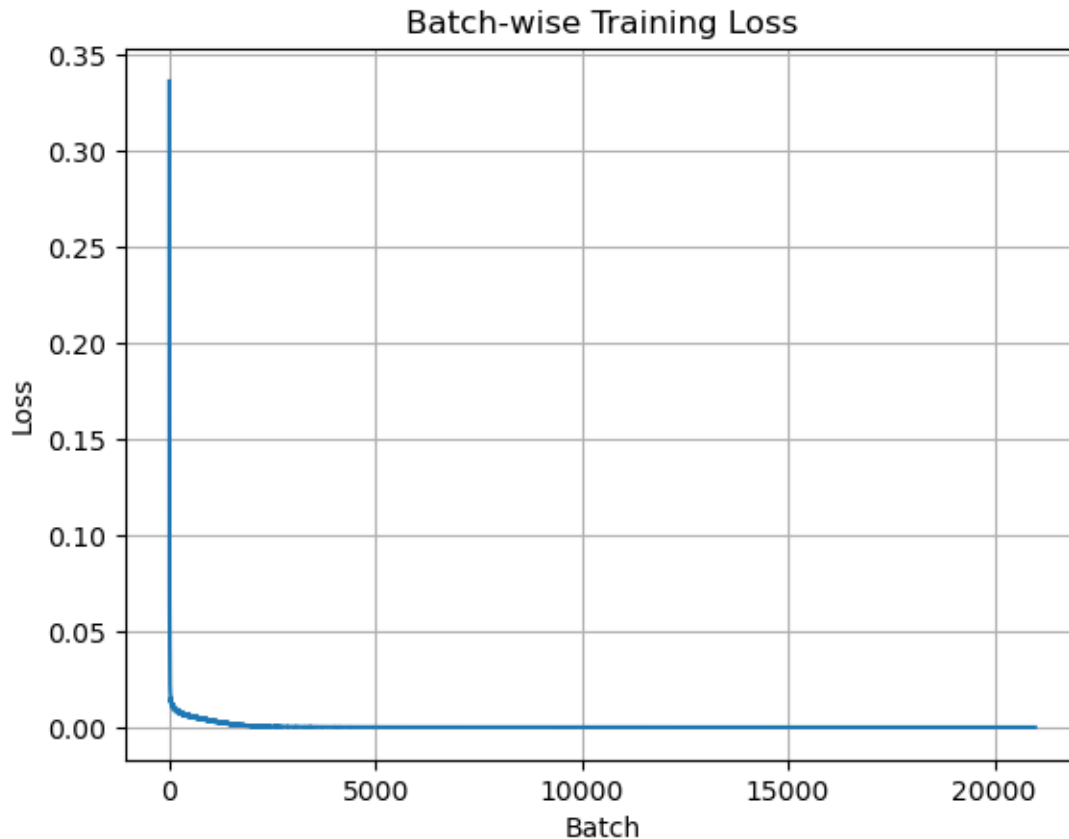
```
[51]: model = CausalTransformer() # Define your model
      losses = train_model(model, train_loader, num_epochs=2, lr=1e-4)
```

```
/opt/anaconda3/lib/python3.11/site-packages/torch/nn/modules/transformer.py:379:
UserWarning: enable_nested_tensor is True, but self.use_nested_tensor is False
because encoder_layer.self_attn.batch_first was not True(use batch_first for
better inference performance)
  warnings.warn(
Using device: cpu
```

Epoch 1 finished - Avg Loss: 0.0010

Epoch 2 finished - Avg Loss: 0.0000

```
[52]: plt.plot(losses)
      plt.title("Batch-wise Training Loss")
      plt.xlabel("Batch")
      plt.ylabel("Loss")
      plt.grid(True)
      plt.show()
```

```
[53]: # Save full model (architecture + weights)
torch.save(model, "causal_transformer.pth")
model
```

```
[53]: CausalTransformer(
  (pos_embedding): Embedding(128, 300)
  (dropout): Dropout(p=0.1, inplace=False)
  (encoder): TransformerEncoder(
    (layers): ModuleList(
      (0-5): 6 x TransformerEncoderLayer(
        (self_attn): MultiheadAttention(
          (out_proj): NonDynamicallyQuantizableLinear(in_features=300,
out_features=300, bias=True)
        )
        (linear1): Linear(in_features=300, out_features=512, bias=True)
        (dropout): Dropout(p=0.1, inplace=False)
        (linear2): Linear(in_features=512, out_features=300, bias=True)
        (norm1): LayerNorm((300,), eps=1e-05, elementwise_affine=True)
        (norm2): LayerNorm((300,), eps=1e-05, elementwise_affine=True)
        (dropout1): Dropout(p=0.1, inplace=False)
      )
    )
  )
```

```

        (dropout2): Dropout(p=0.1, inplace=False)
    )
)
(decoder): TransformerDecoder(
  (layers): ModuleList(
    (0-5): 6 x TransformerDecoderLayer(
      (self_attn): MultiheadAttention(
        (out_proj): NonDynamicallyQuantizableLinear(in_features=300,
out_features=300, bias=True)
      )
      (multihead_attn): MultiheadAttention(
        (out_proj): NonDynamicallyQuantizableLinear(in_features=300,
out_features=300, bias=True)
      )
      (linear1): Linear(in_features=300, out_features=512, bias=True)
      (dropout): Dropout(p=0.1, inplace=False)
      (linear2): Linear(in_features=512, out_features=300, bias=True)
      (norm1): LayerNorm((300,), eps=1e-05, elementwise_affine=True)
      (norm2): LayerNorm((300,), eps=1e-05, elementwise_affine=True)
      (norm3): LayerNorm((300,), eps=1e-05, elementwise_affine=True)
      (dropout1): Dropout(p=0.1, inplace=False)
      (dropout2): Dropout(p=0.1, inplace=False)
      (dropout3): Dropout(p=0.1, inplace=False)
    )
  )
)
(fc_out): Linear(in_features=300, out_features=300, bias=True)
)

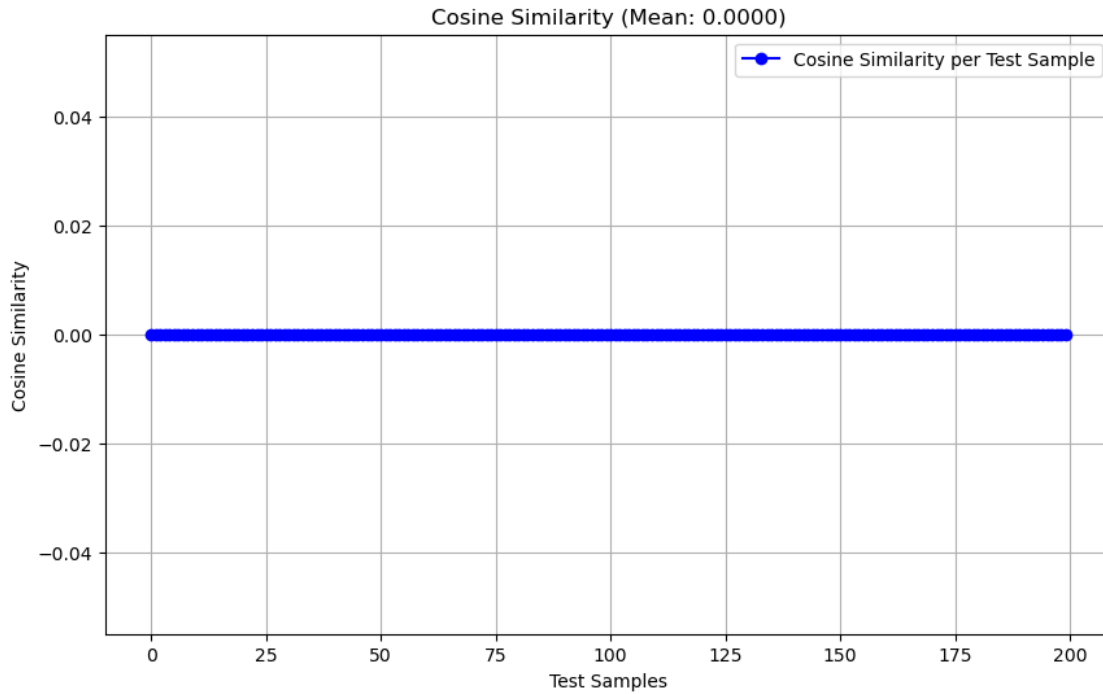
```

```

[54]: avg_cos_sim = evaluate_cosine_similarity(model, test_loader, device,
↳subset_size=200)

print(f"Average Cosine Similarity: {avg_cos_sim:.4f}")

```



Average Cosine Similarity: 0.0000

```
[56]: model = torch.load("causal_transformer.pth")
      model.eval()
      model.to(device)

      tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')

      # Load pretrained BERT model to extract embeddings
      bert_model = AutoModel.from_pretrained("bert-base-uncased")
      bert_model.to(device)
      bert_model.eval()

      projection = torch.nn.Linear(768, 300).to(device)

      def predict_proba(texts):
          tokenized = tokenizer(
              texts,
              padding=True,
              truncation=True,
              max_length=max_len,
              return_tensors='pt'
          ).to(device)

          input_ids = tokenized['input_ids']
```

```

attention_mask = tokenized['attention_mask']

with torch.no_grad():
    bert_outputs = bert_model(input_ids=input_ids,
    ↪attention_mask=attention_mask)
    embeddings = bert_outputs.last_hidden_state
    embeddings = projection(embeddings)

    # Create pos_ids: [0, 1, ..., seq_len-1]
    seq_len = embeddings.shape[1]
    pos_ids = torch.arange(seq_len).unsqueeze(0).expand(embeddings.size(0), -1).
    ↪to(device)

    with torch.no_grad():
        out = model(embeddings, pos_ids)
        final_output = out[:, -1, :] # Last token's output

    probs = torch.softmax(final_output, dim=-1).cpu().numpy()
    return probs

# Instantiate a LimeTextExplainer
explainer = LimeTextExplainer() # Adjust class names to your dataset

# Example text to explain
sample_text = ["The restaurant was amazing!"]

# Use LIME to explain the prediction
explanation = explainer.explain_instance(
    sample_text[0], # Example text to explain
    predict_proba, # Prediction function
    num_features=10 # Number of features to show in the explanation
)

# Visualize the explanation
explanation.show_in_notebook()

```

/var/folders/tz/_kr6c8m90dl2p9klr693qf6w0000gn/T/ipykernel_47991/1850978756.py:1
: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the

loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```
model = torch.load("causal_transformer.pth")
```

<IPython.core.display.HTML object>

Tasks and Comments	Status
Preprocessing Steps - Cleaning, Tokenization, Formatting	Done
Training - Model built with positional embeddings and masked transformer layers	Done
Evaluation - Cosine Similarity	Done
Final AUC value and plot	N/A (Not Applicable Here)
Interpretation using Lime	Done
Next steps Recommended	Optimize: Improve response time using batch processing and model optimizations

Using prompt engineering techniques (such as Few Shot, CoT, DSP etc) on a pre-trained LLM

| Preprocessing Steps - 1. Drop all Nan values 2. Removes spaces, tabs, newlines, or any other invisible characters from the beginning and end of each string| Done | Bibek / Jisna |

" | Training - Model built, train and test handled correctly? | Done | Bibek / Jisna |

" | Evaluation - ROUGE-L Score, BERT Score? | Done. For Few Shots: ROUGE-L: 0.12891, BERT Score: 0.72868 . For COTS: ROUGE-L: 0.07816 / BERT F1: 0.67336| Bibek / Jisna |

" | Interpretation using Lime | Done| Bibek / Jisna |

" | 1st round of tuning - What was the issue faced/tuned? | Done. Tried reducing temperature and top_p for precision but due to limit daily usage of groq, used less dataset for tuning and only performed few shots. ROUGE-L F1: 0.1334 , BERT F1 Score: 0.72876 | Bibek / Jisna |

" | 2nd round of tuning - What was the issue faced/tuned? | Done. Tried increasing slightly temperature and top_p for precision but due to limit daily usage of groq, used less dataset for tuning and only performed few shots. ROUGE-L F1: 0.1337 ,BERT F1 Score: 0.72832 | Bibek / Jisna |

" | Final AUC value? Next steps Recommended? | Done. Increasing the dataset and token size might have improved with the pretrained model | Bibek / Jisna |

```
[1]: !pip install openai>=1.0.0 sentence-transformers bert-score lime scikit-learn
      ↪ tqdm --quiet
```

```
[2]: import pandas as pd
      import numpy as np
```

```
[3]: train_path = '/content/drive/MyDrive/PEDataset/train.csv'
      valid_path = '/content/drive/MyDrive/PEDataset/valid.csv'
```

```
[42]: train_df = pd.read_csv(train_path)
      full_valid_df = pd.read_csv(valid_path)
```

```
[43]: # Preprocessing - clean missing values, strip text
train_df.dropna(subset=['query', 'answers'], inplace=True)
full_valid_df.dropna(subset=['query', 'answers'], inplace=True)
train_df['query'] = train_df['query'].str.strip()
train_df['answers'] = train_df['answers'].str.strip()
full_valid_df['query'] = full_valid_df['query'].str.strip()
full_valid_df['answers'] = full_valid_df['answers'].str.strip()
```

```
[44]: # === Tunable Sample Size ===
# Reduce size for faster testing
VALID_SAMPLE_SIZE = 5000
valid_df = full_valid_df.sample(n=VALID_SAMPLE_SIZE, random_state=42).
    ↪reset_index(drop=True)

# # Phase control: "initial" or "final"
# EVAL_PHASE = "initial"

# # Conditionally reduce size for initial + tuning
# if EVAL_PHASE in ["initial"]:
#     # valid_df = full_valid_df.copy().reset_index(drop=True)
#     VALID_SAMPLE_SIZE = 2000
#     valid_df = full_valid_df.sample(n=VALID_SAMPLE_SIZE, random_state=42).
#         ↪reset_index(drop=True)
# elif EVAL_PHASE in ["final"]:
```

```
[45]: print("Train Data Sample:", train_df.head())
print("Validation Data Sample:", valid_df.head())
```

```
Train Data Sample:                                answers \
0 Kids who are bipolar, in their manic stages, v...
1 Equifax, transunion and experian.
2 Women eat at least 1,200 calories daily and me...
3 Because Caffeine increases the stress hormone ...
4 Kent County

query \
0 why do children get aggressive
1 which credit bureau is used the most for auto ...
2 what is the minimum healthy calorie intake
3 why is coffee making gain weight
4 what county is grand rapids, mi in

finalpassage
0 At the same time, despite claiming the review ...
1 Best Answer: both of those answers are wrong. ...
2 Safe Intakes. If you're not supervised by a me...
3 Is coffee making you fat? If you are overweigh...
4 Located in Grand Rapids, Michigan, the 61st Di...
```

Validation Data Sample:

answers \

```
0 Subcutaneous
1 Employers often want to bring claims for torti...
2 Yes
3 Aegean Sea, Ionian Sea and the Mediterranean Sea.
4 Somatic therapy is treating mental disorders w...
```

query \

```
0 what layer of skin creates heat insulator
1 what convinces employees want
2 can a relation be symmetric and antisymmetric
3 what seas touch greece
4 somatic therapy definition
```

finalpassage

```
0 Superficial heat. In contrast to deep heating ...
1 Whether you want to convince a client to make ...
2 In mathematics, a binary relation R on a set X...
3 The Greek Orthodox Church is an integral part ...
4 Dance therapy or (Dance Movement Psychotherapy...
```

```
[46]: def few_shot_prompt(context, shots):
    prompt = ""
    """
    You are a helpful assistant. Respond appropriately.
    """
    for i in range(min(len(shots), 3)):
        prompt += f"User: {shots.iloc[i]['query']}\nAssistant: {shots.
        ↪iloc[i]['answers']}\n"
    prompt += f"User: {context}\nAssistant:"
    return prompt
```

```
[47]: def cot_prompt(context):
    return f"You are a thoughtful assistant. Think step by step to help users.
    ↪\nUser: {context}\nAssistant: Let's think step by step."
```

```
[49]: # Step 4: Groq Integration using OpenAI-compatible client
import openai
from openai import OpenAI
client = OpenAI(
    ↪api_key="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",
    ↪base_url="https://api.groq.com/openai/v1")
```

```
[51]: def generate_response(prompt, model="llama3-8b-8192"):
    response = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        max_tokens=2000,
        temperature=0.7,
```

```

        top_p=1,
    )
    return response.choices[0].message.content.strip()

```

```

[52]: import time

def safe_generate(prompt, retries=5, delay=10):
    for attempt in range(retries):
        try:
            return generate_response(prompt)
        except Exception as e:
            print(f"[Retry {attempt+1}/{retries}] Error: {e}")
            time.sleep(delay)
    return "Error: Failed after retries"

[53]: # Step 5: Run Inference
from tqdm import tqdm

tqdm.pandas()
few_shot_responses = valid_df['query'].progress_apply(lambda x:
    ↪safe_generate(few_shot_prompt(x, train_df)))

```

```
100%|          | 5000/5000 [1:04:00<00:00, 1.30it/s]
```

```

[54]: cot_responses = valid_df['query'].progress_apply(lambda x:
    ↪safe_generate(cot_prompt(x)))

```

```

70%|          | 3515/5000 [49:16<18:42, 1.32it/s]
[Retry 1/5] Error: no healthy upstream
70%|          | 3520/5000 [49:33<49:33, 2.01s/it]
[Retry 1/5] Error: drop overload
71%|          | 3526/5000 [49:49<35:49, 1.46s/it]
[Retry 1/5] Error: drop overload
[Retry 2/5] Error: drop overload
71%|          | 3538/5000 [50:18<16:56, 1.44it/s]
[Retry 1/5] Error: drop overload
71%|          | 3539/5000 [50:30<1:37:20, 4.00s/it]
[Retry 1/5] Error: drop overload
71%|          | 3542/5000 [50:43<1:25:44, 3.53s/it]
[Retry 1/5] Error: drop overload
71%|          | 3543/5000 [50:56<2:30:06, 6.18s/it]
[Retry 1/5] Error: drop overload

```


100%| | 5000/5000 [1:35:41<00:00, 1.15s/it]

```
[55]: # Step 6: Evaluation
!pip install rouge --quiet
from rouge import Rouge
from bert_score import score
from sentence_transformers import SentenceTransformer
from sklearn.metrics import roc_auc_score

rouge = Rouge()

[56]: def calculate_auc_score(predictions, references):
    model = SentenceTransformer('paraphrase-MiniLM-L6-v2')
    sims = [model.encode(pred, convert_to_tensor=True).dot(model.encode(ref,
↪convert_to_tensor=True)).item()
            for pred, ref in zip(predictions, references)]
    return np.mean(sims)

[57]: print("\nEvaluating Few-Shot...")
fs_hyps, fs_refs = [], []
min_len = min(len(few_shot_responses), len(valid_df['answers']))
for i in range(min_len):
    hyp = str(few_shot_responses.iloc[i]).strip()
    ref = str(valid_df['answers'].iloc[i]).strip()
    if hyp and ref:
        fs_hyps.append(hyp)
        fs_refs.append(ref)
fs_rouge = rouge.get_scores(fs_hyps, fs_refs, avg=True)
P_fs, R_fs, F1_fs = score(fs_hyps, fs_refs, lang="en",
↪model_type="distilbert-base-uncased", device="cuda")
auc_fs = calculate_auc_score(fs_hyps, fs_refs)
```

Evaluating Few-Shot...

```
[58]: print("\nEvaluating CoT...")
cot_hyps, cot_refs = [], []
for i in range(len(cot_responses)):
    if pd.notna(cot_responses.iloc[i]) and pd.notna(valid_df['answers'].
↪iloc[i]):
        cot_hyps.append(str(cot_responses.iloc[i]))
        cot_refs.append(str(valid_df['answers'].iloc[i]))
cot_rouge = rouge.get_scores(cot_hyps, cot_refs, avg=True)
P_cot, R_cot, F1_cot = score(cot_hyps, cot_refs, lang="en",
↪model_type="distilbert-base-uncased", device="cuda")
auc_cot = calculate_auc_score(cot_responses, valid_df['answers'])
```

Evaluating CoT...

```
[59]: print("ROUGE-L (Few-Shot):", fs_rouge['rouge-l'])
      print("BERT F1 (Few-Shot):", F1_fs.mean().item())
      print("AUC (Few-Shot):", auc_fs)
      print("ROUGE-L (CoT):", cot_rouge['rouge-l'])
      print("BERT F1 (CoT):", F1_cot.mean().item())
      print("AUC (CoT):", auc_cot)
```

```
ROUGE-L (Few-Shot): {'r': 0.35956823991219533, 'p': 0.0907374212047726, 'f':
0.12891074328589408}
BERT F1 (Few-Shot): 0.7286812663078308
AUC (Few-Shot): 15.467029666155577
ROUGE-L (CoT): {'r': 0.35016404922283556, 'p': 0.04831503833668224, 'f':
0.07816444747368445}
BERT F1 (CoT): 0.6733647584915161
AUC (CoT): 10.33158920751214
```

```
[60]: # Step 7: Save Results
      valid_df['few_shot_response'] = few_shot_responses
      valid_df['cot_response'] = cot_responses
      valid_df.to_csv('/content/drive/MyDrive/PEDataset/valid_with_responses3.csv',
      ↪index=False)
```

```
[23]: from lime.lime_text import LimeTextExplainer
      from sklearn.pipeline import make_pipeline
      from sklearn.feature_extraction.text import TfidfVectorizer
      from sklearn.linear_model import LogisticRegression

      print("\nRunning LIME on classifier trained with embeddings...")
      X = valid_df['query']
      y = np.concatenate([np.zeros(len(X)//2), np.ones(len(X) - len(X)//2)])
      pipe = make_pipeline(TfidfVectorizer(), LogisticRegression())
      pipe.fit(X, y)
      explainer = LimeTextExplainer(class_names=['Irrelevant', 'Relevant'])
      exp = explainer.explain_instance(X.iloc[0], pipe.predict_proba, num_features=6)
      exp.show_in_notebook(text=True)
```

Running LIME on classifier trained with embeddings...

<IPython.core.display.HTML object>

21 === 1st Empirical Tuning ===

```
[28]: # # Phase control: "initial" or "final"
      # EVAL_PHASE = "final"
      VALID_SAMPLE_SIZE = 2000
```

```

valid_df = full_valid_df.sample(n=VALID_SAMPLE_SIZE, random_state=42).
    ↪reset_index(drop=True)

# # Conditionally reduce size for initial + tuning
# if EVAL_PHASE in ["initial"]:
#     valid_df = full_valid_df.copy().reset_index(drop=True)
# elif EVAL_PHASE in ["final"]:
#     valid_df = full_valid_df.sample(n=VALID_SAMPLE_SIZE, random_state=42).
    ↪reset_index(drop=True)

```

```

[29]: print("\nTuning Round 1: Reducing temperature and top_p for precision")

def generate_response_tuned(prompt, model="llama3-8b-8192"):
    response = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        max_tokens=2000,
        temperature=0.2,
        top_p=0.7,
    )
    return response.choices[0].message.content.strip()

```

Tuning Round 1: Reducing temperature and top_p for precision

```

[30]: def safe_generate1(prompt, retries=5, delay=10):
        for attempt in range(retries):
            try:
                return generate_response_tuned(prompt)
            except Exception as e:
                print(f"[Retry {attempt+1}/{retries}] Error: {e}")
                time.sleep(delay)
        return "Error: Failed after retries"

```

```

[31]: few_shot_tuned = valid_df['query'].progress_apply(lambda x: ↪
    ↪safe_generate1(few_shot_prompt(x, train_df)))

```

```
58%|          | 1155/2000 [13:37<11:18, 1.25it/s]
```

```
[Retry 1/5] Error: drop overload
```

```
58%|          | 1156/2000 [13:50<1:03:40, 4.53s/it]
```

```
[Retry 1/5] Error: drop overload
```

```
100%|         | 2000/2000 [24:51<00:00, 1.34it/s]
```

```

[32]: # Re-evaluate tuned
fs_tuned_hyps = [str(x).strip() for x in few_shot_tuned]
fs_tuned_refs = [str(x).strip() for x in valid_df['answers']]

```

```

fs_tuned_rouge = rouge.get_scores(fs_tuned_hyps, fs_tuned_refs, avg=True)
_, _, F1_fs_tuned = score(fs_tuned_hyps, fs_tuned_refs, lang="en",
    model_type="distilbert-base-uncased", device="cuda")
auc_fs_tuned = calculate_auc_score(fs_tuned_hyps, fs_tuned_refs)

```

```

[33]: print("\n[Post-Tuning Results: Few-Shot]")
print("ROUGE-L:", fs_tuned_rouge['rouge-l'])
print("BERT F1:", F1_fs_tuned.mean().item())
print("AUC:", auc_fs_tuned)

```

```

[Post-Tuning Results: Few-Shot]
ROUGE-L: {'r': 0.3687456367883236, 'p': 0.09525723700687537, 'f':
0.13347133859726443}
BERT F1: 0.7287625074386597
AUC: 15.616145544946194

```

```

[34]: from lime.lime_text import LimeTextExplainer
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

print("\nRunning LIME on classifier trained with embeddings...")
X = valid_df['query']
y = np.concatenate([np.zeros(len(X)//2), np.ones(len(X) - len(X)//2)])
pipe = make_pipeline(TfidfVectorizer(), LogisticRegression())
pipe.fit(X, y)
explainer = LimeTextExplainer(class_names=['Irrelevant', 'Relevant'])
exp = explainer.explain_instance(X.iloc[0], pipe.predict_proba, num_features=6)
exp.show_in_notebook(text=True)

```

Running LIME on classifier trained with embeddings...

<IPython.core.display.HTML object>

22 === 2nd Empirical Tuning ===

```

[35]: print("\nTuning Round 2: Increased temperature and top_p for precision")

def generate_response_tuned2(prompt, model="llama3-8b-8192"):
    response = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        max_tokens=2000,
        temperature=0.4,
        top_p=0.85,
    )

```

```
return response.choices[0].message.content.strip()
```

Tuning Round 2: Increased temperature and top_p for precision

```
[36]: def safe_generate2(prompt, retries=5, delay=10):  
    for attempt in range(retries):  
        try:  
            return generate_response_tuned2(prompt)  
        except Exception as e:  
            print(f"[Retry {attempt+1}/{retries}] Error: {e}")  
            time.sleep(delay)  
    return "Error: Failed after retries"
```

```
[37]: few_shot_tuned2 = valid_df['query'].progress_apply(lambda x:   
    ↪ safe_generate2(few_shot_prompt(x, train_df)))
```

100%| | 2000/2000 [25:39<00:00, 1.30it/s]

```
[38]: # Re-evaluate 2nd tuned  
fs_tuned_hyps = [str(x).strip() for x in few_shot_tuned2]  
fs_tuned_refs = [str(x).strip() for x in valid_df['answers']]  
fs_tuned_rouge2 = rouge.get_scores(fs_tuned_hyps, fs_tuned_refs, avg=True)  
_, _, F1_fs_tuned2 = score(fs_tuned_hyps, fs_tuned_refs, lang="en",   
    ↪ model_type="distilbert-base-uncased", device="cuda")  
auc_fs_tuned2 = calculate_auc_score(fs_tuned_hyps, fs_tuned_refs)
```

```
[39]: print("\n[Post-Tuning Results: Few-Shot]")  
print("ROUGE-L:", fs_tuned_rouge2['rouge-l'])  
print("BERT F1:", F1_fs_tuned2.mean().item())  
print("AUC:", auc_fs_tuned2)
```

```
[Post-Tuning Results: Few-Shot]  
ROUGE-L: {'r': 0.3701380243954667, 'p': 0.0954082618219316, 'f':  
0.13373096099642687}  
BERT F1: 0.7283211350440979  
AUC: 15.560511984199286
```

23 Non Causal Transform

```
[2]: import warnings  
warnings.filterwarnings("ignore")
```

```
[17]: import re  
import contractions  
import numpy as np  
import pandas as pd
```

```
import tensorflow as tf
from transformers import T5Tokenizer
from tensorflow.keras.layers import Input, Embedding, Dense, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
```

```
[20]: # 1. Text Preprocessing
def preprocess_text(text):
    """Clean and normalize text for processing"""
    if not isinstance(text, str):
        return ""
    text = contractions.fix(text) # Expand contractions
    text = text.lower() # Convert to lowercase
    text = re.sub(r'[^a-z\s]', '', text) # Remove special chars/numbers
    text = re.sub(r'\s+', ' ', text).strip() # Remove extra whitespace
    return text
```

```
[44]: train_path = "/Users/kundansingh/Desktop/NLP assignment/untitled folder/train.
    ↪ csv"
valid_path = "/Users/kundansingh/Desktop/NLP assignment/untitled folder/valid.
    ↪ csv"

# 2. Data Loading and Preparation
def load_and_preprocess_data(train_path, valid_path):
    """Load and preprocess training data"""
    train_df = pd.read_csv(train_path)
    valid_df = pd.read_csv(valid_path)

    # Create input-output pairs
    train_data = train_df[["query", "answers"]].rename(columns={"query": "input", "answers": "output"})
    valid_data = valid_df[["query", "answers"]].rename(columns={"query": "input", "answers": "output"})

    # Apply preprocessing
    for df in [train_data, valid_data]:
        df['input'] = df['input'].apply(preprocess_text)
        df['output'] = df['output'].apply(preprocess_text)

    return train_data, valid_data
```

```
[60]: train_data.head()
```

```

[60]:                                     input \
0                                     why do children get aggressive
1  which credit bureau is used the most for auto ...
2          what is the minimum healthy calorie intake
3          why is coffee making gain weight
4          what county is grand rapids, mi in

                                     output
0  kids who are bipolar, in their manic stages, v...
1          equifax, transunion and experian.
2  women eat at least 1,200 calories daily and me...
3  because caffeine increases the stress hormone ...
4          kent county

```

```

[55]: print(train_data.columns)

```

```

Index(['input', 'output'], dtype='object')

```

```

[56]: flattened_data = []

for idx, row in train_data.iterrows():
    human = row['input']
    bot = row['output']

    if isinstance(human, list) and isinstance(bot, list):
        for human_msg, bot_msg in zip(human, bot):
            flattened_data.append({'input': human_msg, 'output': bot_msg})
    elif isinstance(human, str) and isinstance(bot, str):
        # handle single messages
        flattened_data.append({'input': human, 'output': bot})

formatted_df = pd.DataFrame(flattened_data)

```

```

[107]: sampled_df = formatted_df.sample(frac=0.3)
       sampled_df.shape

```

```

[107]: (36000, 2)

```

```

[108]: # Load T5 tokenizer
       from transformers import T5Tokenizer

       tokenizer = T5Tokenizer.from_pretrained("t5-base")

       # Function to tokenize input and output columns for T5
       def tokenize_data_t5(df, tokenizer, max_length=128):
           # Tokenize the input column
           inputs = tokenizer(
               df['input'].tolist(),

```

```

        padding="max_length",
        truncation=True,
        max_length=max_length,
        return_tensors="tf"
    )

    # Tokenize the output column
    outputs = tokenizer(
        df['output'].tolist(),
        padding="max_length",
        truncation=True,
        max_length=max_length,
        return_tensors="tf"
    )

    return inputs, outputs

```

```

[109]: # Tokenize data using T5 tokenizer
tokenized_inputs, tokenized_outputs = tokenize_data_t5(sampled_df, tokenizer)

# Check the tokenized data structure
print("T5 Inputs Shape:", tokenized_inputs['input_ids'].shape)
print("T5 Outputs Shape:", tokenized_outputs['input_ids'].shape)

```

T5 Inputs Shape: (36000, 128)
T5 Outputs Shape: (36000, 128)

```

[110]: import numpy as np
from sklearn.model_selection import train_test_split

# Convert tensors to NumPy arrays for T5 tokenized inputs and outputs
train_inputs_np = tokenized_inputs['input_ids'].numpy()
train_outputs_np = tokenized_outputs['input_ids'].numpy()
attention_masks_np = tokenized_inputs['attention_mask'].numpy()

# Initial train-(validation+test) split (70% train, 30% val+test)
train_inputs, val_test_inputs, train_outputs, val_test_outputs, train_masks, \
    val_test_masks = train_test_split(
        train_inputs_np,
        train_outputs_np,
        attention_masks_np,
        test_size=0.3,
        random_state=42
    )

# Split the (validation+test) set into validation and test sets (15% each)

```



```

val_inputs, test_inputs, val_outputs, test_outputs, val_masks, test_masks = \
    ↪train_test_split(
        val_test_inputs,
        val_test_outputs,
        val_test_masks,
        test_size=0.5,
        random_state=42
    )

# Encoder inputs
train_encoder_inputs = {'input_ids': train_inputs, 'attention_mask': \
    ↪train_masks}
val_encoder_inputs = {'input_ids': val_inputs, 'attention_mask': val_masks}
test_encoder_inputs = {'input_ids': test_inputs, 'attention_mask': test_masks}

# Decoder inputs and outputs for sequence generation (shifted)
train_decoder_inputs = train_outputs[:, :-1]
train_decoder_outputs = train_outputs[:, 1:]

val_decoder_inputs = val_outputs[:, :-1]
val_decoder_outputs = val_outputs[:, 1:]

test_decoder_inputs = test_outputs[:, :-1]
test_decoder_outputs = test_outputs[:, 1:]

```

```

[111]: # Verify shapes and structure
print("Train Encoder Inputs:", train_encoder_inputs['input_ids'].shape,
      "Train Attention Masks:", train_encoder_inputs['attention_mask'].shape)

print("Train Decoder Inputs:", train_decoder_inputs.shape,
      "Train Decoder Outputs:", train_decoder_outputs.shape)

print("Validation Encoder Inputs:", val_encoder_inputs['input_ids'].shape,
      "Validation Attention Masks:", val_encoder_inputs['attention_mask'].shape)

print("Validation Decoder Inputs:", val_decoder_inputs.shape,
      "Validation Decoder Outputs:", val_decoder_outputs.shape)

print("Test Encoder Inputs:", test_encoder_inputs['input_ids'].shape,
      "Test Attention Masks:", test_encoder_inputs['attention_mask'].shape)

print("Test Decoder Inputs:", test_decoder_inputs.shape,
      "Test Decoder Outputs:", test_decoder_outputs.shape)

```

```

Train Encoder Inputs: (25200, 128) Train Attention Masks: (25200, 128)
Train Decoder Inputs: (25200, 127) Train Decoder Outputs: (25200, 127)
Validation Encoder Inputs: (5400, 128) Validation Attention Masks: (5400, 128)
Validation Decoder Inputs: (5400, 127) Validation Decoder Outputs: (5400, 127)

```

Test Encoder Inputs: (5400, 128) Test Attention Masks: (5400, 128)
Test Decoder Inputs: (5400, 127) Test Decoder Outputs: (5400, 127)

```
[112]: import tensorflow as tf

def custom_gelu(x):
    pi = tf.constant(3.141592653589793, dtype=tf.float32) # Define pi
    return 0.5 * x * (1.0 + tf.tanh(tf.sqrt(2.0 / pi) * (x + 0.044715 * tf.
    ↪ pow(x, 3))))
```

```
[113]: import tensorflow as tf

class RelativePositionEncoding(tf.keras.layers.Layer):
    def __init__(self, d_model, max_seq_len, **kwargs):
        super(RelativePositionEncoding, self).__init__(**kwargs)
        self.d_model = d_model
        self.max_seq_len = max_seq_len

    def build(self, input_shape):
        self.relative_positions = self.add_weight(
            shape=(self.max_seq_len, self.d_model),
            initializer="random_normal",
            trainable=True,
            name="relative_positions",
        )

    def call(self, inputs):
        seq_len = tf.shape(inputs)[1] # Get actual sequence length
        return self.relative_positions[:seq_len] # Return relevant slice

    def get_config(self):
        config = super(RelativePositionEncoding, self).get_config()
        config.update({
            "d_model": self.d_model,
            "max_seq_len": self.max_seq_len
        })
        return config
```

```
[114]: from tensorflow.keras.layers import Dense, LayerNormalization, Dropout

class TransformerEncoderLayer(tf.keras.layers.Layer):
    def __init__(self, d_model, num_heads, d_ff, dropout_rate, **kwargs):
        super(TransformerEncoderLayer, self).__init__(**kwargs)
        self.d_model = d_model
        self.num_heads = num_heads
        self.d_ff = d_ff
        self.dropout_rate = dropout_rate
```

```

        # Multi-Head Attention layer
        self.attention = tf.keras.layers.
        ↪MultiHeadAttention(num_heads=num_heads, key_dim=d_model)

        # Feed-forward network
        self.feed_forward = tf.keras.Sequential([
            Dense(d_ff, activation="gelu"), # Use GELU activation
            Dense(d_model)
        ])

        # Normalization layers
        self.layernorm1 = LayerNormalization(epsilon=1e-6)
        self.layernorm2 = LayerNormalization(epsilon=1e-6)

        # Dropout layers
        self.dropout1 = Dropout(dropout_rate)
        self.dropout2 = Dropout(dropout_rate)

    def call(self, x, training, mask=None):
        # Self-attention layer
        attn_output = self.attention(x, x, attention_mask=mask)
        attn_output = self.dropout1(attn_output, training=training)
        out1 = self.layernorm1(x + attn_output) # Add & Norm

        # Feed-forward network
        ffn_output = self.feed_forward(out1)
        ffn_output = self.dropout2(ffn_output, training=training)

        return self.layernorm2(out1 + ffn_output) # Add & Norm

    def get_config(self):
        config = super(TransformerEncoderLayer, self).get_config()
        config.update({
            "d_model": self.d_model,
            "num_heads": self.num_heads,
            "d_ff": self.d_ff,
            "dropout_rate": self.dropout_rate,
        })
        return config

```

```

[115]: import tensorflow as tf
from tensorflow.keras.layers import Dense, LayerNormalization, Dropout

class TransformerDecoderLayer(tf.keras.layers.Layer):
    def __init__(self, d_model, num_heads, d_ff, dropout_rate, **kwargs):
        super(TransformerDecoderLayer, self).__init__(**kwargs)

```

```

self.d_model = d_model
self.num_heads = num_heads
self.d_ff = d_ff
self.dropout_rate = dropout_rate

# Multi-Head Attention layers
self.attention1 = tf.keras.layers.
↳MultiHeadAttention(num_heads=num_heads, key_dim=d_model)
self.attention2 = tf.keras.layers.
↳MultiHeadAttention(num_heads=num_heads, key_dim=d_model)

# Feed-forward network
self.feed_forward = tf.keras.Sequential([
    Dense(d_ff, activation="gelu"), # Use GELU activation
    Dense(d_model)
])

# Normalization layers
self.layernorm1 = LayerNormalization(epsilon=1e-6)
self.layernorm2 = LayerNormalization(epsilon=1e-6)
self.layernorm3 = LayerNormalization(epsilon=1e-6)

# Dropout layers
self.dropout1 = Dropout(dropout_rate)
self.dropout2 = Dropout(dropout_rate)
self.dropout3 = Dropout(dropout_rate)

def call(self, x, enc_output, training, look_ahead_mask=None,
↳padding_mask=None):
    # Self-attention layer (first attention layer)
    attn1 = self.attention1(x, x, attention_mask=look_ahead_mask)
    attn1 = self.dropout1(attn1, training=training)
    out1 = self.layernorm1(x + attn1) # Add & Norm

    # Encoder-decoder attention layer (second attention layer)
    attn2 = self.attention2(out1, enc_output, attention_mask=padding_mask)
    attn2 = self.dropout2(attn2, training=training)
    out2 = self.layernorm2(out1 + attn2) # Add & Norm

    # Feed-forward network
    ffn_output = self.feed_forward(out2)
    ffn_output = self.dropout3(ffn_output, training=training)

    return self.layernorm3(out2 + ffn_output) # Add & Norm

def get_config(self):
    config = super(TransformerDecoderLayer, self).get_config()

```

```

        config.update({
            "d_model": self.d_model,
            "num_heads": self.num_heads,
            "d_ff": self.d_ff,
            "dropout_rate": self.dropout_rate,
        })
    return config

```

```

[116]: from tensorflow.keras.layers import Input, Embedding, Dense
from tensorflow.keras import Model

def build_transformer_model(vocab_size, d_model, num_heads, d_ff,
    ↪ num_enc_layers, num_dec_layers, max_seq_len, dropout_rate):
    # Inputs
    encoder_inputs = Input(shape=(None,), name="encoder_inputs")
    decoder_inputs = Input(shape=(None,), name="decoder_inputs")

    # Embedding and Relative Position Encoding
    embedding = Embedding(vocab_size, d_model)
    relative_position_encoding = RelativePositionEncoding(d_model, max_seq_len)

    # Encoder
    x = embedding(encoder_inputs) + relative_position_encoding(encoder_inputs)
    for _ in range(num_enc_layers):
        x = TransformerEncoderLayer(d_model, num_heads, d_ff, dropout_rate)(x,
    ↪ training=True) # Pass training=True
    encoder_outputs = x

    # Decoder
    y = embedding(decoder_inputs) + relative_position_encoding(decoder_inputs)
    for _ in range(num_dec_layers):
        y = TransformerDecoderLayer(d_model, num_heads, d_ff, dropout_rate)(y,
    ↪ encoder_outputs, training=True) # Pass training=True

    # Final Output
    outputs = Dense(vocab_size)(y)
    return Model(inputs=[encoder_inputs, decoder_inputs], outputs=outputs)

# Corrected arguments
base_model = build_transformer_model(
    vocab_size=vocab_size,
    d_model=128,
    num_heads=8,
    d_ff=512,
    num_enc_layers=2,
    num_dec_layers=2,
    max_seq_len=128, # Corrected position for max_seq_len

```

```

        dropout_rate=0.2    # Corrected position for dropout_rate
    )

```

```

[117]: # Define learning rate scheduler
def scheduler(epoch, lr):
    if epoch < 2:
        return lr
    else:
        return lr * 0.1    # Reduce LR by factor of 10 every 2 epochs

# Initialize the optimizer with gradient clipping
optimizer = Adam(learning_rate=1e-4, clipnorm=1.0)

```

```

[118]: # Corrected build_transformer_model function call
base_model = build_transformer_model(
    vocab_size,
    d_model,
    num_heads,
    d_ff,
    num_enc_layers,
    num_dec_layers,
    max_seq_len,    # Corrected order
    dropout_rate    # Corrected order
)

# Compile the model
base_model.compile(optimizer=optimizer, loss="sparse_categorical_crossentropy")

# Display the model summary
base_model.summary()

```

Model: "functional_28"

Layer (type)	Output Shape	Param #	Connected to
decoder_inputs (InputLayer)	(None, None)	0	-
encoder_inputs (InputLayer)	(None, None)	0	-
embedding_8 (Embedding)	(None, None, 128)	4,096,000	encoder_inputs[0... decoder_inputs[0...
relative_position_... (RelativePositionE...	(None, 128)	16,384	encoder_inputs[0... decoder_inputs[0...

add_10 (Add)	(None, None, 128)	0	embedding_8[0][0... relative_positio...
transformer_encode... (TransformerEncode...	(None, None, 128)	659,712	add_10[0][0]
add_11 (Add)	(None, None, 128)	0	embedding_8[1][0... relative_positio...
transformer_encode... (TransformerEncode...	(None, None, 128)	659,712	transformer_enco...
transformer_decode... (TransformerDecode...	(None, None, 128)	1,187,456	add_11[0][0], transformer_enco...
transformer_decode... (TransformerDecode...	(None, None, 128)	1,187,456	transformer_deco... transformer_enco...
dense_56 (Dense)	(None, None, 32000)	4,128,000	transformer_deco...

Total params: 11,934,720 (45.53 MB)

Trainable params: 11,934,720 (45.53 MB)

Non-trainable params: 0 (0.00 B)

```
[120]: history = base_model.fit(
    [train_inputs, train_decoder_inputs], # Encoder and decoder inputs
    train_decoder_outputs, # Decoder outputs (targets)
    validation_data=([val_inputs, val_decoder_inputs], val_decoder_outputs), #
    ↪ Validation data
    batch_size=32,
    epochs=3,
)
```

```
Epoch 1/3
788/788          708s 899ms/step -
loss: 2.0477 - val_loss: 1.7392
Epoch 2/3
788/788          758s 962ms/step -
loss: 1.7364 - val_loss: 1.7388
Epoch 3/3
```

788/788 803s 1s/step -
loss: 1.7306 - val_loss: 1.7388

```
[121]: # Save the entire model  
base_model.save('m1_model.keras')
```

```
[122]: import tensorflow as tf  
from tensorflow.keras import activations  
  
def custom_gelu(x):  
    return tf.nn.gelu(x) # Or your custom GELU implementation
```

```
[94]: from tensorflow.keras.models import load_model  
  
# Define custom objects (make sure these are defined earlier in the script)  
custom_objects = {  
    "custom_gelu": custom_gelu,  
    "RelativePositionEncoding": RelativePositionEncoding,  
    "TransformerEncoderLayer": TransformerEncoderLayer,  
    "TransformerDecoderLayer": TransformerDecoderLayer  
}  
  
# Load the model  
model = load_model('m1_model.keras', custom_objects=custom_objects)
```

```
[123]: def generate_chatbot_response(model, tokenizer, input_text, max_length=50):  
    """  
    Generate chatbot response using a Transformer encoder-decoder model  
    and HuggingFace T5Tokenizer.  
    """  
  
    # Encode input text for the encoder  
    encoder_input_ids = tokenizer.encode(input_text, return_tensors="tf")  
  
    # Start decoding with just the start token (T5 uses pad_token_id as BOS)  
    decoder_input_ids = tf.constant([[tokenizer.pad_token_id]])  
  
    output_ids = []  
  
    for _ in range(max_length):  
        # Predict next token using current decoder_input_ids  
        predictions = model.predict([encoder_input_ids, decoder_input_ids],  
↪ verbose=0)  
  
        # Get token ID of the most probable next token  
        next_token_id = tf.argmax(predictions[:, -1, :], axis=-1).numpy()[0]  
  
        # Stop if end-of-sentence token (</s>) is generated
```



```

    if next_token_id == tokenizer.eos_token_id:
        break

    output_ids.append(next_token_id)

    # Append predicted token to decoder inputs
    decoder_input_ids = tf.concat([decoder_input_ids, [[next_token_id]]], ↵
↵axis=-1)

    # Decode output token IDs to string
    decoded_response = tokenizer.decode(output_ids, skip_special_tokens=True)
    return decoded_response

```